

青沐生命科技微信小程序

代码审查与重构设计文档

代码审查报告 · 重构架构设计 · 产品原型与业务建议 · 开发任务拆解

生成日期 2026-06-18 · 基于 running-group-stats 全量代码审查

青沐生命科技小程序 • 重构文档包

生成日期：2026-06-10 | 基于代码库 running-group-stats 全量审查（15 页面 / 13 云函数 / ~8800 行）

文档导航

文档	内容	读者
01-code-review.md	代码审查报告：P0/P1/P2 问题清单、各文件速查表	全体开发
02-architecture.md	重构架构：目录结构、数据库集合、云函数 API 契约、登录/群/支付流程	开发（实现依据）
03-product-prototype.md	产品原型与业务建议：业务闭环、竞品参考、逐页交互说明	产品 + 开发
04-task-breakdown.md	任务拆解：5 个 Phase、验收标准、里程碑、风险登记	项目负责人 + 开发
05-payment.md	支付接入：微信支付/支付宝申请方法与材料、开发细则、参考代码、积分内部化规则	负责人（申请）+ 开发
06-device-integration.md	手表/手环对接：蓝牙 BLE 实时心率 + 各厂商平台 OAuth 授权数据采集（Phase 6 任务表）	开发
07-food-nutrition-apis.md	国内菜谱/营养 API 选型：菜谱内容、营养成分、AI 菜品识别、缓存代理设计	产品 + 开发
08-recipe-ingestion-and-ludong.md	菜谱采集 ETL 管道（统一 Schema/去重/审核）+ 内部律动平台双向对接规范（Phase 7）	开发 + 律动团队
09-code-optimization.md	代码优化清单（重扫）：性能/包体/去重/可维护性，含 before/after 与优先级	开发
review-package.html	全部文档的网页汇总版（可浏览器打开/打印）	任何人
review-package.pdf	PDF 版（可打印/分发）	任何人

30 秒结论

- 1. 现状是可演示原型：约 80% 功能为模拟实现，不可直接上线。

- 2. 三个致命问题必须先修：钱包余额客户端可篡改（P0-1）、云函数信任前端 openid（P0-2/3）、"自动统计微信群消息"无微信开放能力支撑（P0-6，需按 02 § 5.2 的 shareTicket+打卡方案重做）。
- 3. 支付依赖商户号审批，钱包/会员购买/支付全部挂功能开关，V1.0 用"积分兑换 + 意向单"先行上线。
- 4. 重构总量约 28-35 人天，按 04 文档 Phase 0→5 执行，Phase 0 地基不完成不开发业务。

01 代码审查报告

项目：青沐生命科技微信小程序（running-group-stats）审查日期：2026-06-10 | 审查范围：15 个页面、13 个云函数、全局配置，共约 8800 行 结论：当前代码是「可演示的原型」，不是「可上线的产品」。约 80% 的业务逻辑为模拟实现（mock），且存在多个安全级别的致命缺陷，不建议在现有代码上直接补功能，应按 02 号文档的架构做受控重构。

1. 总体评估

维度	评分 (满分5)	说明
可运行性	★★☆☆☆	云环境 ID 为占位符、sitemap.json 缺失、3 个被调用的云函数不存在
安全性	★☆☆☆☆	openid 由前端传入可伪造；钱包余额前端计算可任意篡改
技术可行性	★★☆☆☆	核心卖点「自动统计微信群聊消息」无对应微信开放能力，前提不成立
代码质量	★★☆☆☆	大量 mock 数据硬编码、重复逻辑、3565 行单文件样式、无封装
产品完整度	★★★★☆	页面骨架和信息架构基本齐全，UI 结构清晰，可作为原型参考

模拟实现占比统计（"假功能"清单）：

模块	真实程度
登录	假 — 未调用 wx.login，拿不到 openid，全靠 'test_openid' 兜底
群绑定/自动统计	假 — bindToGroup 用 Math.random() 生成假群名（group.js: 120-148）
群成员排名/周月年汇总	假 — 张三李四王五硬编码（group.js:3-50, 380-657）
商城（statistics 页）	假 — 8 个商品硬编码，下单只弹 Toast（statistics.js:148-160）
会员购买	

模块	真实程度
	假 — 确认弹窗后直接写本地缓存，无支付（settings.js:152-178）
钱包充值	假 — setTimeout 1 秒后本地加余额（wallet.js:96-140）
马拉松报名/酒店预订/马博会购买	假 — 调用了不存在的云函数
AI 机器人/智能体	假 — KIMI、即梦、抖音发布全部为 console.log 模拟
用户资料、运动APP绑定、打卡入库	半真 — 云函数有真实数据库读写，但 openid 链路是断的

2. P0 问题（安全/阻断，必须修复才能上线）

P0-1 钱包余额由前端计算并整体覆盖写库 —— 可任意篡改资金

- 位置：pages/wallet/wallet.js:96-140（doRecharge）、cloudfunctions/save-wallet-data/index.js
- 现状：前端 balance + amount 后调用 save-wallet-data 把 balance、transactions 整体写入数据库；云函数不做任何校验。
- 风险：任何用户用调试工具调用云函数即可把自己余额改成任意数字。涉及资金的字段绝不允许客户端直写。
- 修复方向：余额只能由服务端在支付回调/消费扣减中变更（见 02 文档 §6 钱包设计）。当前支付商户号未下来，整个钱包模块应挂功能开关隐藏。

P0-2 所有云函数信任前端传入的 openid —— 身份可伪造

- 位置：全部 13 个云函数（如 save-checkin/index.js:11、get-wallet-data/index.js:11）
- 现状：const { openid } = event，前端传什么就是谁。任何人可读写他人的资料、钱包、打卡记录。
- 修复方向：云函数内一律使用 cloud.getWXContext().OPENID，删除 event 中的 openid 参数。这是云开发的标准做法，一行代码的事。

P0-3 'test_openid' 兜底导致全部用户数据混写

- 位置：10 个页面文件共 14 处，如 index.js:88、group.js:573、wallet.js:48 的 userInfo.openId || 'test_openid'
- 现状：getUserProfile 返回的 userInfo 根本没有 openid 字段，所以所有真实用户都会落到 'test_openid'，全员共享同一份资料/钱包/打卡数据。

- 修复方向：随 P0-2 一并消灭；openid 永远不从前端传。

P0-4 登录链路断裂：从未调用 `wx.login`

- 位置：`pages/login/login.js`、`app.js`
- 现状：登录页只调 `getUserProfile` 拿头像昵称就跳首页；没有 `code` → `code2Session` / 云函数换取 openid 的步骤；无登录态、无路由守卫，未登录也能用全部功能。
- 附带问题：`getUserProfile` 自 2022-10-25 起已回收，返回匿名头像昵称；`getUserInfo` (`app.js:18`、`login.js:33`) 同样废弃。需改用 `button open-type="chooseAvatar" + input type="nickname"` 方案。

P0-5 调用不存在的云函数（运行时必然报错）

- `save-marathon-registration` (`marathon.js:118`)
- `save-hotel-booking` (`hotel.js:97`)
- `save-purchase` (`marathon-expo.js:191`)
- 三个页面的"成功" Toast 都在云函数调用之外提前弹出，用户看到"报名成功"但什么都没发生。

P0-6 「自动统计微信群聊消息」技术前提不成立

- 位置：`group.js` (`startAutoStats/checkGroupMessages`)、`cloudfunctions/ai-robot`、`get-group-data`、`send-summary`
- 事实：微信不向小程序/云函数开放任何读取群聊消息的 **API**，也不能主动向微信群发消息。`ai-robot` 的 `getGroupMessages()` 和 `send-summary` 的 `sendMessageToGroup()` 永远只能是 mock。
- 可行替代（见 02 文档 §5）：
- 群身份识别：群内转发小程序卡片 → `wx.getGroupEnterInfo` 拿 `opengid`，实现"同群成员看同一份榜单"；
- 数据来源：成员在小程序内打卡（已有 `save-checkin`），而非抓群消息；
- 周报触达：订阅消息（`subscribe message`）推给个人 + 生成战报图片让群主转发到群。

P0-7 基础配置占位符 / 缺失文件

- `app.js:8`：`env: 'your-cloud-env-id'` → 所有 `wx.cloud.callFunction` 失败。
- `app.js:35`：`baseUrl: 'https://your-backend-api.com'`（实际无人使用，删除）。
- `index.js:46`：百度地图 AK 为占位符 `'您的百度地图AK'`；且百度 geocoder **v2** 已停服，`request` 合法域名也未配置。建议改用腾讯位置服务（微信系，有官方小程序 SDK）。
- `app.json:55` 引用 `sitemap.json`，文件不存在，构建报错。
- 13 个云函数目录均无 **package.json**，`wx-server-sdk` 依赖未声明，无法部署。
- `app.json:54`：`"debug": true` 上线前必须关闭。

3. P1 问题（功能缺陷）

#	问题	位置	说明
P1-1	积分由前端计算并上传，可作弊	group.js:530、update-user-points	<code>Math.floor(distance)</code> 在前端算好传给云函数；应服务端按打卡记录计算，并加防作弊上限（如单日 $\leq 50\text{km}$ ）
P1-2	打卡无任何数据校验	group.js:516、save-checkin	距离可为负数/9999，配速格式不校验，可重复提交刷分
P1-3	<code>setInterval</code> 自动统计无意义且泄漏	group.js:170-176	小程序切后台即挂起，定时器不可靠；switch 反复开关会注册多个 interval 且从不清理
P1-4	<code>wx.openUrl</code> 不存在	smart-agent.js:78	小程序无此 API，打开外链需 web-view 或复制链接
P1-5	onPullDownRefresh 永不触发	index.js:104	页面未配置 <code>enablePullDownRefresh</code> （项目里根本没有任何页面级 .json 文件）；且用 <code>this.onLoad()</code> 模拟刷新是反模式
P1-6	钱包流水存在单文档数组里	save-wallet-data	云数据库单文档上限 1MB，流水必须独立集合
P1-7	退出登录逻辑无效	settings.js:130-145	删除的 <code>userInfo</code> storage 从未被写入过；且 <code>navigateTo</code> 到 login 后用户按返回又回来了，应 <code>reLaunch</code>
P1-8	首页/我的页数据全部硬编码	index.wxml:60-80、settings.wxml:18-35	"128 次打卡 / 365 公里 / 1234 积分"为写死数字，与数据库无关；首页"平均配速 42"语义错误
P1-9	会员等级规则双处重复且不一致风险	group.js:155-168、settings.js:33-60	等级→权益映射各写一份，改一处漏一处
P1-10	图片资源缺失	get-group-data 等引用 /images/avatar.png	项目无 images 目录

4. P2 问题（代码质量/工程规范）

1. 页面命名与功能不符：`statistics` 实为商城、`group` 实为运动中心、`settings` 实为"我的"。tabBar 文案（首页/运动/商城/我的）与目录名对不上，新人接手必然踩坑。重构时目录改名为 `mall`、`sport`、`mine`。

2. **app.wxss 3565** 行单文件，无页面级 wxss、无设计变量（主色 #1aad19 是微信绿，与"青沐"品牌无关，需定品牌色）。
3. 无任何封装：`wx.cloud.callFunction` 散落 20+ 处，`success/fail` 回调风格混杂 `async/await`；无统一 `loading`、错误提示、重试。
4. 无组件化：排行榜项、商品卡片、功能列表项等重复结构应抽 `component`。
5. 无 **.gitignore**：`project.private.config.json`、`.DS_Store` 已入库。
6. **statistics.wxml 528** 行：分类/品牌/详情三层视图挤在一页靠 `wx:if` 切换，应拆页面或组件。
7. **console.log** 残留 40+ 处；注释里大量"实际项目中这里应该…"——这些就是本报告的待办清单。
8. **smart-agent** 模块定位存疑：在 C 端小程序里放"AI 生成视频并发布抖音"的运营工具，且抖音发布无合规 API，建议移出小程序（见 03 文档 §7）。

5. 各文件问题速查表

文件	P0	P1	P2	重构动作
app.js	环境ID占位、废弃 API	—	baseUrl 无用	重写（登录态管理）
app.json	sitemap 缺失	debug:true	tabBar 无图标	修正
app.wxss	—	—	3565行	拆分到页面/组件
pages/login	无 wx.login	废弃 getUserProfile	—	重写
pages/index	百度AK/已停服API	假数据、下拉刷新失效	—	重写数据层，UI 保留
pages/group	群消息前提不成立、test_openid	定时器泄漏、打卡无校验、积分作弊	657行混3个职责	拆为 sport + group 两页，按新群方案重写
pages/statistics	—	假商品、假下单	528行wxml	改名 mall，接数据库
pages/settings	假会员购买	退出登录无效、假数据	等级规则重复	改名 mine，会员购买挂支付开关
pages/wallet	余额可篡改	流水单文档	—	

文件	P0	P1	P2	重构动作
				服务端重写 + 功能开关
pages/profile	test_openid	实名认证为假	showModal 编辑体验差	数据链路修复，认证接微信实名能力或暂下线
pages/bind-app	test_openid	绑定为假（无 OAuth）	—	暂改为"敬请期待"或仅记录意向
pages/marathon、hotel、food、scenic、rural-support、marathon-expo	云函数不存在	全假数据	结构雷同	统一为"内容/服务"模块，数据进数据库（见 02 § 7）
pages/smart-agent	抖音发布不可行	wx.openUrl 不存在	—	移出 C 端，独立评估
cloudfunctions/*	openid 信任前端、无 package.json	—	init 风格不一	按 02 § 8 合并重写为 6 个函数

6. 做得对的地方（保留）

- 选择微信云开发（免运维、免域名备案），适合当前团队规模，继续沿用。
- 页面信息架构完整，四个 tab 的产品骨架（首页/运动/商城/我的）方向正确。
- wxml 结构和 class 命名整洁一致，UI 层可大量复用。
- `update-user-points` 用了 `db.command.inc` 原子自增，写法正确。
- `save` 类云函数普遍有"查询→存在则更新否则新建"的 `upsert` 意识。

下一步请阅读：**02-architecture.md**（目标架构与数据库/API 设计）→ **04-task-breakdown.md**（按此分配开发任务）。

02 重构架构设计

项目：青沐生命科技微信小程序 本文档是重构的唯一技术依据，开发同学按本文实现，遇到与旧代码冲突处一律以本文为准。产品定位（已确认）：大健康生活方式平台 = 运动社群 + 健康/运动商城 + 赛事与本地服务。钱包/支付因商户号申请中，全部挂功能开关。

1. 设计原则

1. 服务端权威：身份（openid）、积分、余额、订单状态只能由云函数产生和变更，前端永远只是展示与发起。
2. 能力边界内设计：不依赖微信未开放的能力（读群消息、向群发消息、抖音发布）。
3. 功能开关：未就绪的模块（钱包、支付、会员购买、智能体）通过远程配置隐藏，不删代码不堵路。
4. 渐进式重构：保留云开发技术栈与现有 UI 骨架，按模块逐个替换数据层，避免推倒重来。
5. 单一数据源：每条业务规则（会员权益、积分规则）只在一处定义（服务端配置 + 前端 constants 镜像）。

2. 总体架构



wallet	余额/流水/支付(开关关闭时全部拒绝)	
admin	内容管理/配置下发(仅管理员openid白名单)	
云数据库(集合见 §4) 云存储(商品图/头像/战报图) 定时触发器(周报聚合)		

为什么合并成 6 个云函数：13 个零散函数 → 每个函数冷启动、部署、权限都要单独管。按领域合并 + `event.action` 路由，是云开发社区的成熟实践，也方便共用鉴权与校验中间层。

3. 前端目录结构（目标）

```

miniprogram/
├─ app.js / app.json / app.wxss (仅设计变量+通用类<300行)
├─ sitemap.json                ← 必须补建
├─ config/
│   └─ env.js                  // 云环境ID、版本号
├─ utils/
│   ├── auth.js                // 登录态：ensureLogin()、getUser()、logout()
│   ├── format.js              // 配速/距离/日期格式化
│   └─ constants.js            // 会员等级、商品分类等枚举（与服务端配置对齐）
├─ services/                    // 唯一允许出现 callFunction 的地方
│   ├── api.js                 // call(name, action, data) 统一封装：loading/错误toast/登录态过期
│   ├── user.js sport.js mall.js content.js wallet.js
├─ components/
│   ├── ranking-list/ product-card/ cell/ empty-state/ feature-gate/
├─ pages/
│   ├── index/                  // 首页 (tab1)
│   ├── sport/                  // 运动中心 (tab2, 原 group 改名拆分)
│   ├── group-detail/           // 群榜单详情
│   ├── mall/                   // 商城 (tab3, 原 statistics 改名)
│   ├── product-detail/         // 商品详情（从 528 行 wxml 中拆出）
│   ├── order-confirm/ order-list/
│   ├── mine/                   // 我的 (tab4, 原 settings 改名)
│   ├── profile/ bind-app/ wallet/ membership/
│   └─ content-list/ content-detail/ // 赛事/酒店/景区/餐饮/乡村 五合一模板页
└─ images/tabbar/*.png          // tabBar 图标（当前缺失）

```

改名映射：statistics→mall、group→sport、settings→mine。马拉松/酒店/美食/景区/乡村振兴五个页面合并为 **content-list + content-detail** 模板页，用 `type` 参数区分，消灭五份雷同代码。

4. 数据库设计（云数据库集合）

所有集合权限设为「仅创建者可读写」或「所有用户不可读写（仅云函数）」，禁止前端直连写库。

users (用户, merge 现 users/user_details)

```
{
  _openid, nickname, avatarFileID,
  profile: { name, phone, gender, birthday, region, height, weight },
  certified: false,           // 实名认证 (二期)
  memberLevel: 'free|monthly|quarterly|yearly',
  memberExpireAt: Date|null,  // 会员到期时间 (服务端写)
  points: 0,                  // 积分余额 (仅云函数 inc)
  boundApps: { garmin:false, huawei:false, ... },
  stats: { totalDistance, totalCheckins, totalPoints }, // 冗余汇总, 打卡时 inc
  createdAt, updatedAt
}
```

checkins (打卡记录)

```
{
  _openid, groupId|null,
  distance: Number,           // km, 服务端校验 0.5~50
  durationSec: Number|null,   // 建议新增: 时长, 可反推配速防伪造
  pace: 'mm:ss', heartRate, cadence,
  points: Number,             // 服务端计算
  date: 'YYYY-MM-DD',        // 用于"当日重复打卡"约束 (同日同群最多1次计积分)
  createdAt
}
```

groups / group_members (跑群)

```
// groups
{ _id, opengid|null, name, ownerOpenid, memberCount, createdAt }
// group_members
{ groupId, _openid, nickname, avatarFileID, joinedAt, role:'owner|member' }
```

products / orders (商城)

```
// products
{ _id, name, category, brand, price, originalPrice, memberDiscount,
  images:[fileID], description, stock, status:'on|off', sort }
// orders
{ _id, _openid, items:[{productId,name,price,qty}], totalAmount,
  pointsUsed, payAmount,
  status:'pending_pay|paid|shipped|done|cancelled', // 支付开关关闭时只允许 pending_pay→cancelled
  payment:{ transactionId, paidAt }|null, address, createdAt }
```

points_records (积分流水, 只增不改)

```
{ _openid, change:+10|-100, type:'checkin|signup_bonus|order_deduct|member_gift',
  refId, balance:Number, createdAt }
```

wallets / wallet_transactions (钱包, 开关关闭期仍建好结构)

```
// wallets:{ _openid, balance:0, status:'active|frozen', updatedAt }
// wallet_transactions:{ _openid, type:'recharge|consume|refund', amount,
//   orderId|null, wxTransactionId|null, status:'success|pending|failed', createdAt }
```

铁律: balance 只由 wallet 云函数在「微信支付回调验证成功」或「订单扣减」时变更; 任何来自 event 的 balance 字段直接拒绝。

contents (赛事/酒店/景区/餐饮/乡村振兴 统一内容表)

```
{ _id, type:'marathon|hotel|scenic|food|rural',
  title, cover:fileID, summary, detail(富文本),
  price|fee, date|validRange, location, tags:[],
  actionType:'enroll|book|link|none', // 报名/预订/外链/纯展示
  status:'on|off', sort, createdAt }
```

enrollments (报名/预订意向)

```
{ _openid, contentId, type, formData:{name,phone,...}, status:'submitted|confirmed|cancelled'
```

支付未开通前, 马拉松报名/酒店预订一律走「意向登记+客服跟进」, 不收钱。

app_config (远程配置/功能开关, 所有用户只读)

```
{ _id:'feature_flags',
  wallet:false, payment:false, membershipPurchase:false,
  smartAgent:false, bindApp:false,
  noticeBar:'', minVersion:'1.0.0' }
{ _id:'member_levels', free:{maxGroups:2,discout:1}, monthly:{price:29.9,maxGroups:5,discoun
{ _id:'points_rules', perKm:1, dailyMaxKm:50, dailyMaxCheckins:1, signupBonus:50 }
```

5. 关键流程设计

5.1 登录（替换现有假登录）

```
小程序启动 app.onLaunch
├ wx.login() → code (其实云函数不需要code, 云开发自动注入openid)
├ call user.login {}
  云函数: openid = cloud.getWXContext().OPENID
           users 中无记录 → 创建(送 signupBonus 积分) ; 有 → 返回用户+config
├ 本地缓存 user + featureFlags ; globalData.user 就绪
首次完善资料 (可跳过) :
  button open-type="chooseAvatar" → 头像临时文件 → 上传云存储
  input type="nickname" → 昵称
  call user.updateProfile
```

- 登录页不再是闸口：游客可浏览首页/商城/内容，打卡、下单、进群时 `auth.ensureLogin()` 拦截补登录。
- 废弃 `getUserProfile/getUserInfo` 全部删除。

5.2 跑群（替代「读取微信群消息」的可行方案）

```
创建群：群主在 sport 页点「创建跑群」→ call sport.createGroup → 得 groupId
入群：群主点「邀请」→ wx.shareAppMessage 转发到微信群（携带 groupId）
      成员点卡片进入 → 若由群聊打开，wx.getGroupEnterInfo 可取 openid
      （openid 用于绑定"小程序群↔微信群"，同一微信群只允许建一个跑群）
      call sport.joinGroup {groupId}
打卡：成员在小程序内填距离/配速（或后续接微信运动步数）→ call sport.checkin
      服务端：校验范围/当日次数 → 写 checkins → inc 用户积分 → inc 群统计
榜单：call sport.groupRanking {groupId, period:'week|month|year'}
      聚合 checkins (按 date 范围 group by openid)
周报：云函数定时触发器(每周日20:00) 聚合各群数据 → 写 group_reports
      → 订阅消息推给已订阅成员 + 生成战报分享图，群主一键转发回微信群
```

- 会员等级决定可加入/创建群数量（free 2 / monthly 5 / quarterly 8 / yearly 15，读 `app_config`）。
- `setInterval`、`ai-robot`、`send-summary`、`get-group-data` 全部删除。

5.3 打卡积分（防作弊）

服务端规则（`points_rules` 配置）： $\text{distance} \in [0.5, 50]$ ；每日计分打卡 ≤ 1 次；积分 = $\text{floor}(\text{distance} \times \text{perKm})$ ；流水写 `points_records`，余额 `db.command.inc`。前端只展示结果，不上传 `points` 字段——云函数收到 **points** 字段直接忽略。

5.4 下单（支付开关两态）

payment=false（现阶段）：

商品页可加购物车、可生成订单(pending_pay) → 提示「支付功能开通中，可用积分全额兑换或联系客服」

积分兑换：payAmount==0 且 pointsUsed 足够 → 直接 paid（服务端扣积分）

payment=true（商户号下来后）：

mall.createOrder → wallet.unifiedOrder（云开发 cloudPay.unifiedOrder）

→ 前端 wx.requestPayment → 支付回调云函数验签 → 订单 paid + 写流水

5.5 会员购买

同 5.4：开关关闭时「立即开通」按钮显示「敬请期待」；开通后走微信支付，服务端写 memberLevel + memberExpireAt，删除现在写 **localStorage** 的假购买。

6. 钱包与微信支付接入（给负责人看的申请清单）

1. 在「微信公众平台 → 微信支付」申请商户号（需营业执照、对公账户，约 1-5 个工作日）。
2. 关联 AppID wx426885831a05f18e，开通 JSAPI 支付。
3. 云开发控制台绑定商户号后即可用 cloud.cloudPay.unifiedOrder（免证书、免回调域名，最省事）。
4. 上线顺序建议：先开「商品直接支付」，缓上「余额充值」——余额充值涉及预付卡资质（单用途预付卡需备案），合规成本高。如无强需求，砍掉"充值余额"，保留"积分 + 微信支付"双轨即可。
5. 开关切换：管理员改 app_config.feature_flags.payment=true，前端无需发版。

7. 云函数 API 契约（前后端对齐用）

统一调用：wx.cloud.callFunction({ name, data: { action, payload } }) 统一返回：

{ code: 0, data } | { code: 4xx/5xx, msg }（前端 api.js 统一弹错）

函数	action	payload	返回 data	备注
user	login	—	{ user, config }	首次自动注册
user	updateProfile	{ nickname?, avatarFileID?, profile? }	{ user }	字段白名单过滤
user	bindApps	{ boundApps }	{ boundApps }	flag bindApp 关闭时返回 403
sport	checkin	{ distance, durationSec?,	{ points, todayDone }	服务端算分

函数	action	payload	返回 data	备注
		pace?, heartRate?, cadence?, groupId? }		
sport	myStats	{ period }	{ totalDistance, count, avgPace }	聚合 checkins
sport	createGroup / joinGroup / quitGroup	{ name } / { groupId, openid? }	{ group }	数量上限按会员级
sport	groupRanking	{ groupId, period }	{ members:[...], totals }	
mall	listProducts	{ category?, brand?, keyword?, page }	{ list, total }	
mall	productDetail	{ id }	{ product }	
mall	createOrder	{ items, address, pointsUsed }	{ orderId, payAmount }	
mall	myOrders / cancelOrder	{ page } / { orderId }	{ list } / { }	
content	list	{ type, page }	{ list }	五类内容统一
content	detail / enroll	{ id } / { id, formData }	{ content } / { enrollmentId }	enroll 仅登记意向
wallet	get / unifiedOrder / transactions	...	...	flags.wallet=false 时一律 {code: 403, msg: '功能开通中'}
admin	upsertContent / upsertProduct / setConfig	...	...	openid 白名单校验

每个云函数目录必须有 **package.json** ("wx-server-sdk": "~2.6.3" 或最新) , 入口统一模板 :

```
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
exports.main = async (event) => {
  const { OPENID } = cloud.getWXContext() // 唯一身份来源
  const { action, payload = {} } = event
  try { return { code: 0, data: await handlers[action]({ OPENID, payload }) } }
```

```
catch (e) { return { code: e.code || 500, msg: e.message } }  
}
```

8. 工程规范

- **app.json**：补 sitemap.json；debug 删除；tabBar 配图标（iconPath/selectedIconPath）；新增页面按 §3 注册；考虑分包（mall 相关页一个分包，content 一个分包）。
- 品牌：主色不再用微信绿 #1aad19，建议青沐品牌青绿色系（如 #0FAF8E，由设计定稿），在 app.wxss 顶部定义 `page { --brand: ...; --brand-light: ...; }` 全局变量。
- **.gitignore**：project.private.config.json、.DS_Store、node_modules/、cloudfunctions/**/package-lock.json。
- 错误处理：services/api.js 统一 showLoading/hideLoading、code !== 0 时 toast msg、网络失败重试一次。
- 日志：删除全部 console.log，云函数保留 console.error（云开发有日志检索）。

9. 渐进式迁移步骤（与 04 文档任务对应）

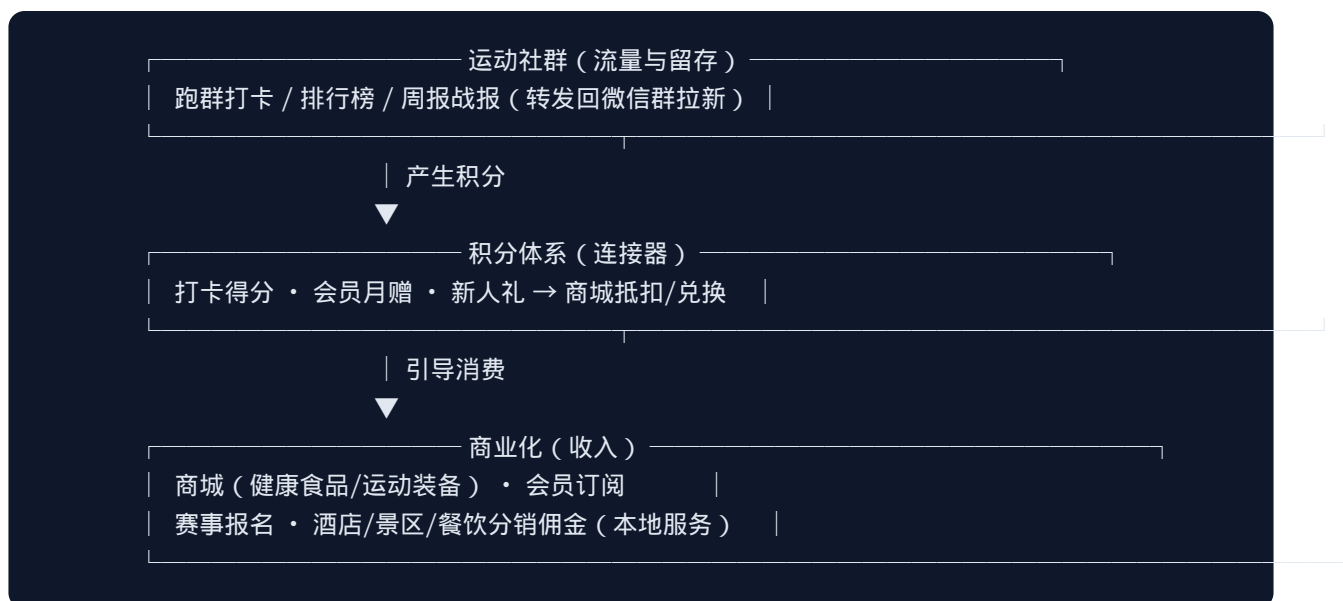
1. 打地基：补 sitemap/.gitignore/package.json、真实云环境 ID、api.js 封装、app_config 集合 + feature-gate 组件。
2. 换身份：user 云函数 + 新登录流，删除一切 test_openid 与 event.openid。
3. 运动闭环：checkin 服务端化 → 群创建/加入/榜单 → 周报订阅消息。
4. 商城真数据：products/orders 集合 + admin 录入 + 积分兑换通路。
5. 内容五合一：contents 集合迁移五个静态页 + 意向登记。
6. 支付接入（等商户号）：打开开关，会员购买 + 订单支付 + 钱包。
7. 暂缓：smart-agent 移出小程序、bind-app 等第三方 OAuth、实名认证。

03 产品原型说明与业务建议

项目：青沐生命科技微信小程序 定位（已确认）：大健康生活方式平台 —— 运动社群（跑群打卡）+ 健康/运动商城 + 赛事与本地服务。本文回答三件事：业务怎么跑（含竞品参考，应负责人要求给出建议）、页面长什么样、每页交互规则。

1. 业务模型建议

1.1 核心闭环：运动 → 积分 → 消费



为什么这样设计：跑群打卡是高频行为（日活抓手），商城和服务是低频高价值行为（变现），积分是把高频导向低频的桥。微信群战报转发是零成本获客渠道——每周一张带小程序码的群战报图，就是裂变素材。

1.2 竞品参考（建议研究对象）

竞品	可借鉴点	与我们的差异
Keep	会员订阅 + 自营商城 + 课程内容的变现组合；勋章/等级激励	它是独立 App 重内容，我们是微信生态轻社群
悦跑圈	跑团运营、线上马拉松（报名费+完赛奖牌电商）模式	线上马拉松是成熟的"赛事+电商"变现，强烈建议二期做
咕咚	运动数据接硬件/第三方 App 的生态打法	对应我们的 bind-app 模块（建议二期）
马拉马拉 / 最酷	马拉松赛事报名聚合 + 周边服务（酒店、火车票）	正是 content 模块（赛事+酒店+景区）的对标
薄荷健康	大健康商城选品（代餐、营养补剂）+ 健康档案	青沐"生命科技"背景可向健康品类发力，毛利高于运动装备

差异化定位建议："微信群里的跑团助手 + 大健康严选"。不做大而全的运动 App，吃透两点：① 微信群是中国跑团的真实组织形态，工具长在群里；② 依托青沐供应链做健康品类商城（运动营养、康养产品），避开与 Keep 拼装备。

1.3 分期上线建议

阶段	内容	商业目标
V1.0 (先上)	登录 + 打卡 + 跑群榜单/周报 + 商城 (积分兑换/意向单) + 内容展示	跑通日活, 积累用户
V1.5 (商户号下来)	微信支付: 商品购买、会员订阅、赛事报名收费	开始收入
V2.0	线上马拉松、微信运动步数接入、订阅消息运营、分销佣金	规模化
暂缓/独立	smart-agent (AI 生成抖音内容是运营内部工具, 不该放在 C 端小程序, 且抖音无合规自动发布 API)、第三方运动 App OAuth、实名认证	—

2. 信息架构 (IA)

TabBar

- 首页 index 快捷入口 + 我的数据概览 + 活动banner + 公告
- 运动 sport 今日打卡 | 我的跑群列表 → 群详情(榜单/周报/邀请)
- 商城 mall 分类/搜索 → 商品列表 → 商品详情 → 下单
- 我的 mine 资料/会员/积分/订单/钱包(开关)/绑定/客服/设置

二级页

- group-detail 群榜单(周/月/年) · 成员 · 邀请 · 群设置(群主)
- product-detail / order-confirm / order-list
- membership 会员对比与开通(开关)
- wallet 余额/流水(开关, 默认隐藏)
- profile 资料编辑 · bind-app 运动APP绑定(开关)
- content-list / content-detail 赛事 · 酒店 · 景区 · 餐饮 · 乡村振兴(type参数)

3. 关键流程图

3.1 登录与游客模式

启动 → user.login(静默, 云端openid) → 拉取 feature_flags
游客可: 逛首页/商城/内容
需登录动作(打卡/下单/进群/会员)
 └─ ensureLogin() — 未完善资料 —> 弹窗: 微信头像昵称授权(新接口) —> 继续原动作

3.2 跑群生命周期

群主: 运动页[创建跑群] → 填群名 → 生成群卡片 → 转发到微信群
成员: 点击卡片(群聊场景) → `getGroupEnterInfo` → `openid` → [加入跑群] → 进入群详情
打卡: 运动页填 距离/时长(配速自动算) → 服务端校验+记分 → 榜单实时更新
周报: 每周日20:00 定时聚合 → 订阅消息通知成员 → 群主[生成战报图]转发回微信群
退群/解散: 成员可退; 群主解散需二次确认

3.3 购买流程 (双态)

商品详情 → [加入购物车]/[立即购买] → 订单确认(地址/积分抵扣)
`payment=OFF`: 仅当 积分可全额抵扣 → 兑换成功
否则 → 「支付开通中」意向单 + 客服企微二维码
`payment=ON`: `wx.requestPayment` → 回调验签 → 已支付 → 我的订单跟踪

4. 页面原型说明

标注规则: 【保留】沿用现有 UI; 【改】调整; 【新】新增。数据来源一律为 § 02 文档 API, 不再硬编码。

4.1 首页 index【改】

- 顶部: 问候 + 头像昵称 (未登录显示"点击登录") + 公告条 (`app_config.noticeBar`, 可关闭)。
- 服务推荐宫格【保留6宫格】: 马拉松、赛事权益、酒店、景区、餐饮、乡村振兴 → `content-list?type=x`。图标用真实 icon 替换 emoji。
- 数据概览【改】: 改为我的真实数据 (本周跑量、连续打卡天数、积分余额、所在群数), 来自 `sport.myStats + user`; 未登录显示引导卡片。删除"平均配速42"这类无意义占位。
- 最新活动【改】: 读 `contents` 中 `status=on` 按 `sort` 取 2 条; 「查看更多」→ `content-list`。
- 移除: 进入即获取定位并入库 (改为内容页需要城市筛选时再申请定位, 减少授权打扰与隐私审查风险)。

4.2 运动 sport (tab, 原 group 页拆分)【改】

- 区块1 今日打卡: 距离(必填)、时长(必填, 自动算配速)、心率/步频(选填)。提交后按钮变"今日已打卡 ✓" (服务端 `todayDone`)。表单校验: 0.5-50km。
- 区块2 我的跑群: 群卡片列表 (群名、人数、我的本周名次) → 点击进 `group-detail`; 底部「创建跑群」「加入跑群(输入邀请码/通过卡片)」; 显示 "已加入 n/上限 m (会员可提升)"。

- 删除：原页面的成员排名/汇总/群设置/发送汇总（全部移入 group-detail），原 657 行一页三职责的问题随之解决。

4.3 群详情 group-detail【新】

- 头部：群名、人数、（群主）编辑/邀请按钮（shareAppMessage 带 groupId）。
- Tab：周榜 | 月榜 | 年榜。榜单项：名次(前三高亮)、头像、昵称、公里数、次数、平均配速。
- 周报卡片：最近一期战报摘要 + 「生成战报图」（canvas 绘制带小程序码长图，引导转发）。
- 群主管理：改名、移除成员、解散群。订阅消息开关（引导 requestSubscribeMessage）。

4.4 商城 mall（tab，原 statistics 改名）【改】

- 搜索框（关键词→ mall.listProducts）+ 分类 chips（健康食品/运动营养/装备/周边…分类从 app_config 下发）。
- 商品卡片：图(云存储)、名称、价格、会员价标签、积分可抵标识。
- 悬浮购物车入口（角标数量）。
- 商品详情 product-detail【新，从 528 行 wxml 拆出】：轮播图、价格区（划线价/会员价）、详情/评价 tab（评价二期）、底部栏[客服][购物车][立即购买]。
- 下单 order-confirm【新】：地址(profile.address，无则填写)、积分抵扣滑块（上限=可用积分与订单额取小）、支付按钮按 payment 开关两态（见 3.3）。

4.5 我的 mine（tab，原 settings 改名）【改】

- 头部：头像昵称 → profile；数据条：累计跑量/打卡/积分（真实数据）。
- 会员卡片：当前等级+到期日；「开通/续费」→ membership 页（payment=OFF 时按钮文案"敬请期待"）。权益对比表数据来自 app_config.member_levels，不再前端写死。
- 功能列表：我的订单、我的报名、积分明细【新】、收货地址、运动APP绑定(开关)、钱包(开关，默认隐藏入口)、联系客服(open-type="contact" 接微信客服，替换写死的400电话)、设置。
- 退出登录：清 storage + reLaunch 到首页（游客态），修复现有无效逻辑。

4.6 钱包 wallet【改，默认隐藏】

- feature_flags.wallet=false：mine 页不显示入口；直达路径显示「钱包功能正在申请开通，敬请期待」占位页（feature-gate 组件统一处理）。
- 开通后：余额（只读，服务端值）、流水分页列表（wallet_transactions）、充值按钮视合规决定是否提供（见 02 § 6 第4条——建议砍掉充值，保留支付+积分）。
- 删除：现有"绑定支付宝/储蓄卡/信用卡"假绑定 UI（小程序内不可能绑这些，误导用户）。

4.7 内容页 **content-list / content-detail**【新，五合一】

- list：按 type 渲染（marathon/hotel/scenic/food/rural），卡片含封面、标题、标签、价格/日期；城市筛选（此时才申请定位）。
- detail：富文本详情 + 底部动作按钮按 actionType：报名/预订(意向表单：姓名+手机号)| 外链(复制链接)| 纯展示。
- 提交意向 → 「已收到，工作人员将联系您」+（可选）客服企微二维码。

4.8 登录/资料 **login·profile**【改】

- 不再有独立"登录闸口页"；login 页改造为"完善资料"半屏弹窗组件：chooseAvatar 按钮 + nickname 输入框 + 手机号(可选，button open-type="getPhoneNumber" 需认证主体)。
- profile：列表式编辑（姓名/性别/生日/地区/身高/体重/地址），用 picker 和表单页替代现在的 showModal 输入（showModal editable 体验差且不支持 picker 类型）。实名认证入口隐藏（二期）。

5. 设计规范要点

- 品牌色：弃用微信绿 #1aad19，改青沐青绿系（建议主色 #0FAF8E / 深 #0B8C72 / 浅底 #E6F7F3，最终以 VI 为准）；全局 CSS 变量管理。
- tabBar：补 4 组图标（普通/选中），文案：首页 · 运动 · 商城 · 我的。
- 空态/加载/错误三态组件全局统一（empty-state 组件）。
- 所有 emoji 图标替换为 iconfont 或切图（emoji 在不同机型渲染不一致）。
- 合规：隐私协议弹窗（首启）、《用户协议》《隐私政策》链接进 mine-设置；商城类目需在小程序后台开通「百货/食品」类目，健康食品需相应资质。

6. 订阅消息模板申请清单（提前在小程序后台申请）

用途	模板类型	触达时机
周报通知	数据统计提醒	每周日 20:00
打卡提醒	运动提醒	用户自选时间（二期）
订单发货通知	物流提醒	发货时
报名结果通知	报名审核结果	客服确认后

7. 明确不做 / 暂缓清单（避免开发资源浪费）

1. 读取/监听微信群聊天记录 —— 微信无此能力，永远不做（用打卡+战报替代）。
2. 小程序内主动向微信群发消息 —— 同上，用「生成战报图引导转发」替代。
3. **smart-agent AI** 发抖音 —— 移出小程序；若运营确需 AI 内容工具，做内部 Web 工具单独立项。
4. 余额充值 —— 涉及预付资质合规，V1.5 不做，用「积分 + 微信支付直付」覆盖场景。
5. 第三方运动 **App OAuth**（佳明/华为等） —— 各家开放平台申请周期长，V2 评估；V1 入口挂"敬请期待"。
6. 实名认证 —— 等有强场景（线上马拉松领奖）再接。

04 开发任务拆解

项目：青沐生命科技微信小程序重构 用法：按 Phase 顺序执行；每条任务含验收标准（AC），完成即勾选。工作量单位：人天（1 名熟悉小程序的开发）。依赖文档：01 审查报告（问题编号 P0-x/P1-x 对应）、02 架构（§ 引用）、03 原型。总量估算：V1.0 约 **28-35** 人天（不含 UI 切图与商品/内容素材准备）。

Phase 0 地基修复（约 3 天）—— 不做完不允许开发任何业务

ID	任务	对应问题	工作量	验收标准 (AC)
T0-1	创建 sitemap.json；app.json 删除 "debug": true	P0-7	0.5	开发者工具编译 0 报错 0 警告（资源类除外）
T0-2	开通/确认云开发环境，app.js 填真实 env ID；删除无用 baseUrl	P0-7	0.5	任一云函数可调通并返回
T0-3	每个云函数补 package.json（wx-server-sdk），init 统一 DYNAMIC_CURRENT_ENV	P0-7	0.5	全部云函数可一键部署成功
T0-4	新增 .gitignore（project.private.config.json/.DS_Store/node_modules），并从 git 移除已入库文件	P2-5	0.5	git status 干净
T0-5	建 services/api.js 统一封装（action 路由、loading、错误 toast、code 约定见 02 § 7）	P2-3	1	任意页面 3 行代码完成一次云函数调用并自动处理错误

ID	任务	对应问题	工作量	验收标准 (AC)
T0-6	建 app_config 集合 + feature_flags 文档 + components/feature-gate 组件	02 § 4	1	改库里 wallet=false , 钱包入口即消失 (无需发版)

Phase 1 身份与用户 (约 4 天)

ID	任务	对应	工作量	AC
T1-1	user 云函数 : login (getWXContext 取 openid , 首登建档+注册积分)、updateProfile、字段白名单	P0-2/3/4 , 02 § 5.1	1.5	① event 传入伪造 openid 不生效 ② 新用户首登 users 集合自动建档 ③ 全库无 'test_openid' 写入
T1-2	前端 utils/auth.js : ensureLogin、登录态缓存、游客模式拦截	02 § 5.1	1	游客可浏览 ; 点打卡/下单弹出完善资料流程
T1-3	资料弹窗组件 : chooseAvatar + nickname 新接口 ; 删除全部 getUserProfile/getUserInfo	P0-4	1	真机可设置头像昵称 ; 代码 grep 无废弃 API
T1-4	profile 页改造 : picker/表单页编辑 , 对接 user.updateProfile ; 实名认证入口隐藏	03 § 4.8	0.5	各字段可编辑保存、刷新后仍在
T1-5	mine 页 (settings 改名) : 真实数据头部、退出登录改 reLaunch、客服改 open-type="contact"	P1-7/8	0.5	退出后回首页游客态 ; 数据来自接口

Phase 2 运动闭环 (约 8 天) ★ 产品核心

ID	任务	对应	工作量	AC
T2-1	sport.checkin : 服务端校验 (0.5-50km、当日 1 次计分)、算积分、写 checkins + points_records、inc users.points/stats	P1-1/2 , 02 § 5.3	1.5	① 传 points 字段被忽略 ② 距离 -1/999 被拒 ③ 当日第 2 次打卡不加分

ID	任务	对应	工作量	AC
T2-2	sport 页 (group 改名重做) : 打卡表单 (时长→自动配速)、今日已打卡态、我的群列表; 删除 <code>setInterval</code> /发送汇总/假成员	P0-6 , P1-3 , 03 § 4.2	1.5	打卡后立即见积分增长; 页面无任何写死人名
T2-3	<code>sport.createGroup/joinGroup/quitGroup</code> : 上限按 <code>app_config.member_levels</code> ; <code>shareAppMessage</code> 邀请卡; 群聊场景 <code>getGroupEnterInfo</code> 绑定 <code>opengid</code>	02 § 5.2	2	两个真机账号可建群、经卡片入群; 超上限被拦截并提示升级会员
T2-4	group-detail 页: 周/月/年榜 (<code>sport.groupRanking</code> 聚合)、成员管理 (群主)	03 § 4.3	1.5	榜单数字与 <code>checkins</code> 手工核对一致
T2-5	周报: 定时触发器聚合 <code>group_reports</code> + 订阅消息推送 + 战报分享图 (<code>canvas</code> 含小程序码)	02 § 5.2 , 03 § 6	1.5	周日 20:00 收到订阅消息; 战报图可保存转发
T2-6	index 首页数据化: <code>myStats</code> 概览、公告条、活动位读 <code>contents</code> ; 移除启动定位	P1-5/8 , 03 § 4.1	1	首页无硬编码数字; 启动不再弹定位授权

Phase 3 商城与内容 (约 8 天)

ID	任务	对应	工作量	AC
T3-1	<code>products</code> 集合 + <code>admin.upsertProduct</code> (<code>openid</code> 白名单) + 商品图传云存储	02 § 4/ § 7	1	管理员可增改商品并即时生效
T3-2	mall 页 (<code>statistics</code> 改名重做) : 分类/搜索/列表分页 (<code>mall.listProducts</code>)	03 § 4.4	1.5	8 个 mock 商品删除; 搜索分类可用
T3-3	product-detail 独立页 (拆 528 行 <code>wxml</code>)	P2-6	1	详情页可分享、可加购
T3-4	购物车 (本地 <code>storage</code>) + <code>order-confirm</code> + <code>mall.createOrder</code> (积分抵扣, <code>payment=OFF</code> 双态逻辑见 02 § 5.4)	03 § 3.3	2	① 积分足额可 0 元兑换成功且服务端扣分 ② 不足时显示"支付开通中"意向单
T3-5	<code>order-list</code> 我的订单 + <code>cancelOrder</code>	02 § 7	0.5	订单状态流转正确
T3-6			2	

ID	任务	对应	工作量	AC
	contents 集合 + content-list/content-detail 五合一模板页；删除 marathon/hotel/food/scenic/rural-support/marathon-expo 六个旧页及其对不存在云函数的调用	P0-5， 03 § 4.7		五类内容同一套页面渲染；报名意向落 enrollments 表

Phase 4 支付与钱包（约 5 天，前置条件：商户号审批通过）

ID	任务	前置	工作量	AC
T4-1	商户号关联 AppID + 云开发绑定（负责人办理，见 02 § 6）	营业执照	—	cloudPay 沙箱可下单
T4-2	wallet.unifiedOrder + 支付回调验签 + 订单置 paid + 流水落库；重写 save/get-wallet-data ，余额禁止客户端写（删除现有实现）	P0-1	2	篡改回调/伪造金额被拒；账实一致
T4-3	membership 页 + 会员购买支付化，删除 settings.js 写 storage 的假购买；服务端写 memberLevel/到期/月赠积分定时任务	P0-4 相关	1.5	购买后等级、群上限、商城折扣即时生效；到期自动降级
T4-4	打开 payment/wallet/membershipPurchase 开关，回归测试 3.3 双态	T4-2/3	1	开关 ON/OFF 两态均通过验收用例
T4-5	钱包页重做：余额只读 + 流水分页；删除"绑定支付宝/银行卡"假 UI	P1-6， 03 § 4.6	0.5	流水来自 wallet_transactions 集合

Phase 5 收尾与上线（约 3 天）

ID	任务	工作量	AC
T5-1	品牌换色 + tabBar 图标 + emoji→iconfont + 空态/加载组件统一	1	设计走查通过
T5-2	隐私协议弹窗 + 用户协议/隐私政策页 + 小程序后台类目与资质（食品类目）	0.5	审核要求项齐备
T5-3	数据库权限复查：所有集合「仅云函数可写」；admin 白名单配置	0.5	控制台直接写库被拒

ID	任务	工作量	AC
T5-4	删除遗留：smart-agent 页面与云函数、ai-robot、get-group-data、send-summary（被 T2-5 替代）、save-user-location、全部 console.log	0.5	grep 无 console.log；云函数仅剩 02 § 7 的 6 个
T5-5	体验版全流程回归（用例：游客浏览/注册/打卡/建群入群/榜单/兑换/意向单/退出）+ 提审	0.5	用例全过，提审通过

里程碑与并行建议

周1 周2 周3 周4 周5(等商户号)
Phase0→Phase1 → Phase2 → Phase3 → Phase4 → Phase5上线
└ 前后端可并行：A同学做云函数(T1-1,T2-1,T2-3)，B同学做页面(T1-2~5,T2-2)
Phase4 与 Phase3 之间无强依赖，商户号何时下来何时插入。

风险登记

风险	影响	缓解
商户号审批延期	会员/支付收入延后	V1.0 以积分兑换 + 意向单先行，不阻塞上线
群聊场景 opengid 需用户授权且仅群聊打开有效	群绑定体验有门槛	兜底"邀请码入群"路径（T2-3 已含）
订阅消息一次性授权限制	周报触达率低	打卡成功页顺势请求订阅授权；战报图转发兜底
食品类商品资质	商城审核被拒	提前在后台开通类目并备齐资质（T5-2 前置办理）
旧体验版已有测试数据	脏数据混入	上线前清空 checkins/users 等集合

05 支付接入设计：申请指南 + 开发细则 + 参考代码

项目：青沐生命科技微信小程序 本文档供：负责人（办理申请）+ 开发（按细则实现）。参考代码仅为文档内容，不直接入库，实现时以 02 文档 API 契约为骨架。积分定位（已确认）：积分

是纯内部虚拟激励，仅由系统内部计算发放与抵扣，不充值、不提现、不转赠、不与人民币双向兑换，不涉及任何实际结算交易。

0. 先说结论（决定申请什么）

渠道	能否用在微信小程序内	结论
微信支付	✓ 唯一选择	必须申请，本项目全部线上收款走它
支付宝支付	✗ 不能	微信与支付宝生态互相隔离，微信小程序内无法调起支付宝（包括内嵌 H5、二维码等变通方式均被支付协议限制）。支付宝申请仅在未来做支付宝小程序 / 自有 App / 独立 H5 商城时才需要，本文 §3 仍给出完整申请方法备用
积分	—	内部记账，不是支付渠道，见 §5

1. 微信支付申请（负责人办理清单）

1.1 前置条件

1. 小程序主体必须是企业或个体工商户（个人主体永远开不了支付）。
2. 小程序已完成微信认证（微信公众平台提交企业资料，认证费 300 元/年）。
3. 有对公银行账户（企业必须对公；个体工商户可用经营者银行卡）。账户名称必须与营业执照主体名称一致。

1.2 所需材料清单

材料	要求
营业执照	有效期内，照片/扫描件清晰，统一社会信用代码
法定代表人身份证	正反面照片；非法人办理另需经办人身份证+授权函
对公账户信息	开户行全称、开户支行、账号（用于打款验证）
联系信息	经办人手机号、邮箱（接收审核通知）
经营信息	商户简称（会显示在用户账单）、客服电话、经营类目、门店/网站/小程序截图

材料	要求
特殊类目资质	商城卖食品/保健品需《食品经营许可证》等；卖医疗器械需相应许可。青沐若上架健康食品，提前做好

1.3 申请步骤（线上全流程，约 1-5 个工作日）

1. 登录 [微信支付商户平台](#) → 「接入微信支付」→ 选择「小程序支付」场景进件。
2. 填写主体信息、上传材料 → 提交审核（1-2 个工作日）。
3. 审核通过后：账户验证（微信向对公账户打一笔随机金额，回填确认）+ 法人/经办人签约。
4. 拿到 商户号（**mch_id**）后，在商户平台「产品中心 → AppID 账号管理」绑定小程序 **AppID**（wx426885831a05f18e），小程序管理员扫码确认授权。
5. 商户平台「API 安全」：设置 **APIv3** 密钥、下载/申请 **API** 证书（云开发 cloudPay 模式可跳过证书，见 § 4.1）。
6. （本项目推荐）云开发控制台 → 「微信支付」→ 绑定该商户号，之后云函数可免证书调用支付。

1.4 费率与到账

- 标准费率 **0.6%**（按经营类目 0.6%–1% 不等，民生类目可能更低）；T+1 自动结算到对公账户。
- 申请下来后：管理员把 `app_config.feature_flags.payment` 改为 `true`，前端无需发版即可放开支付（见 02 文档 § 6）。

2. 支付宝申请（备用，未来多端使用）

再次强调：这些能力用不到微信小程序里，适用对象是未来的支付宝小程序、自有 App、独立 H5 商城、线下收款码。

2.1 所需材料

与微信支付高度一致：营业执照、法人身份证、对公账户（或个体户经营者支付宝账户）、经营类目说明、网站/App/小程序信息；食品类目同样需要许可证。另需一个企业支付宝账户（用营业执照注册，收款入账用）。

2.2 申请步骤

1. 注册 [支付宝开放平台](#) 开发者账号（用企业支付宝登录）→ 完成企业实名认证。

2. 开放平台控制台创建应用：未来做支付宝小程序则「创建小程序」；做 App/H5 则「创建网页/移动应用」，拿到 **APPID**。
3. 「产品绑定」按场景开通支付产品并签约（审核约 1-3 个工作日）：

产品	适用场景
小程序支付（JSAPI）	支付宝小程序内收款
手机网站支付（WAP）	独立 H5 商城
APP 支付	自有 App
当面付	线下扫码/收款码（活动现场收报名费可用）

1. 配置密钥：开放平台「开发设置」生成 应用公私钥（**RSA2**），上传应用公钥、保存支付宝公钥（或用证书模式）。
2. 费率：一般 **0.38%-0.6%**（当面付低、线上标准 0.6%，类目和活动期有浮动），T+1 结算。

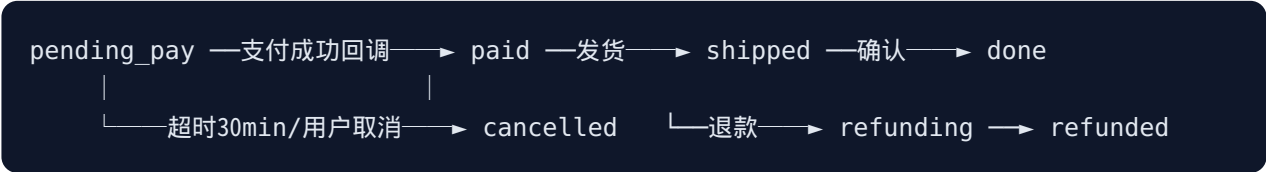
2.3 本项目的建议

V1.x 阶段不投入支付宝开发；待商城在微信端验证跑通、确有多端需求（如线下马拉松现场收费）再启动，优先级排在 V2 之后。

3. 支付总体设计（两渠道统一抽象）



订单状态机（orders.status，只允许如下迁移，其余一律拒绝）：



开发细则（两渠道通用，逐条验收）：

1. 金额：全链路用整数「分」，禁止浮点；客户端传来的金额只作展示，服务端按商品表现价重算，不一致即拒单。

2. 幂等：`outTradeNo` = 订单ID，回调按 `outTradeNo` 加锁处理；重复回调直接返回成功不重复记账。

3. 验签：回调必须验签（cloudPay 模式

由平台代验：仅信任云开发投递的回调；APIv3 模式验证 Wechatpay-Signature 头）。严禁以前端回报的"支付成功"作为发货依据，唯一依据是服务端回调/主动查单。4. 超时：下单时记 `expireAt=30min`，定时函数将过期 `pending_pay` 置 `cancelled` 并释放库存。5. 对账：每日定时函数拉取账单（`cloudPay downloadBill` / 支付宝对账单接口），与 `orders` 比对，差异写告警表。6. 退款：仅管理员白名单可发起，按原路退回，记 `refund` 流水，先退积分抵扣部分→再退现金部分。7. 日志：支付链路每一步落 `pay_logs` 集合（脱敏，不存证书/密钥）。

4. 微信支付参考代码（云开发 **cloudPay** 方案，本项目首选）

选 `cloudPay` 的理由：免 API 证书、免回调域名备案、回调直接投递到指定云函数，最适合云开发架构。若未来迁出云开发，再换 APIv3 直连（§4.4）。

4.1 下单（云函数 **wallet**，**action=unifiedOrder**）

```
// cloudfunctions/wallet/pay.js
const cloud = require('wx-server-sdk')
cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
const db = cloud.database()

async function unifiedOrder({ OPENID, payload }) {
  const { orderId } = payload
  const flags = (await db.collection('app_config').doc('feature_flags').get()).data
  if (!flags.payment) throw { code: 403, message: '支付功能开通中' }

  const order = (await db.collection('orders').doc(orderId).get()).data
  if (order._openid !== OPENID) throw { code: 403, message: '无权操作' }
  if (order.status !== 'pending_pay') throw { code: 409, message: '订单状态不可支付' }

  // 服务端重算金额（分），绝不信任前端
  const payAmount = await recalcAmount(order) // 商品现价×数量 - 积分抵扣
  if (payAmount !== order.payAmount) throw { code: 409, message: '价格已变动，请重新下单' }

  const res = await cloud.cloudPay.unifiedOrder({
    body: '青沐商城-订单' + orderId.slice(-6),
    outTradeNo: orderId, // 幂等键
    spbillCreateIp: '127.0.0.1',
    subMchId: process.env.MCH_ID, // 商户号放环境变量，不写死
    totalFee: payAmount, // 单位：分
    envId: cloud.DYNAMIC_CURRENT_ENV,
    functionName: 'payCallback' // 回调云函数
  })
  return res.payment // 含 timeStamp/nonceStr/package/paySign
}
```

4.2 支付回调（云函数 **payCallback**，唯一记账入口）

```
// cloudfunctions/payCallback/index.js
exports.main = async (event) => {
  // cloudPay 投递的 event 含 returnCode/resultCode/outTradeNo/totalFee/transactionId
  if (event.returnCode !== 'SUCCESS' || event.resultCode !== 'SUCCESS')
    return { errcode: 0, errmsg: 'OK' } // 失败回调也要应答，避免重投

  const orderId = event.outTradeNo
  await db.runTransaction(async t => {
    const order = (await t.collection('orders').doc(orderId).get()).data
    if (order.status === 'paid') return // 幂等：重复回调直接放过
    if (order.status !== 'pending_pay') throw new Error('状态异常')
    if (event.totalFee !== order.payAmount) throw new Error('金额不符') // 防篡改

    await t.collection('orders').doc(orderId).update({ data: {
      status: 'paid',
      payment: { transactionId: event.transactionId, paidAt: new Date() }
    }})
    await t.collection('wallet_transactions').add({ data: {
      _openid: order._openid, type: 'consume', amount: event.totalFee,
      orderId, wxTransactionId: event.transactionId, status: 'success', createdAt: new Date()
    }})
  })
  // 会员订单：在此处写 memberLevel/memberExpireAt（服务端唯一入口）
  return { errcode: 0, errmsg: 'OK' } // 必须按此格式应答
}
```

4.3 前端调起（**services/wallet.js** + 页面）

```
// services/wallet.js
import { call } from './api'
export async function payOrder(orderId) {
  const payment = await call('wallet', 'unifiedOrder', { orderId })
  await wx.requestPayment(payment) // 用户输密码/指纹
  // 注意：requestPayment 成功 ≠ 入账成功，最终状态以轮询订单为准
  return pollOrderStatus(orderId) // 轮询 3 次×2s 查 orders.status === 'paid'
}
```

4.4 备选：**APIv3** 直连要点（迁出云开发时再用）

商户平台下载证书 + 设置 APIv3 密钥 → 服务端用官方 SDK（如 wechatpay-node-v3）调

JSAPI 下单 接口拿 prepay_id → 自行用商户私钥生成 paySign 返回前端 → 回调接口验证

Wechatpay-Signature + AES-256-GCM 解密报文。其余状态机/幂等/对账细则与 §3 完全一致。

4.5 支付宝参考代码骨架（未来 H5/支付宝小程序用）

```
// 服务端 (Node) : alipay-sdk
const AlipaySdk = require('alipay-sdk').default
const alipay = new AlipaySdk({ appId, privateKey, alipayPublicKey })
// 下单 (H5 手机网站支付)
const url = alipay.pageExec('alipay.trade.wap.pay', {
  bizContent: { out_trade_no: orderId, total_amount: '29.90',
    subject: '青沐商城订单', product_code: 'QUICK_WAP_WAY' },
  notify_url: 'https://api.xxx.com/alipay/notify'
})
// 回调 : alipay.checkNotifySign(postData) 验签 → 同 §4.2 的幂等记账逻辑
```

5. 积分规则（内部计算，零结算风险）

定性：积分是运营激励工具，等同“虚拟勋章+优惠抵扣额度”，不是预付卡、不是虚拟货币。守住四条合规红线：

- 1. 只进不提：积分可获得、可抵扣（最多抵订单的 X%，建议 50%，或全额兑换指定“积分专区”商品），不可充值购买、不可提现、不可转赠、不可兑换现金。
- 2. 单向折算：仅在抵扣瞬间按固定比率折算（如 100 积分 = 1 元抵扣额），系统内不存在“人民币 → 积分”的购买通道（会员月赠积分是权益赠送，不是购买）。
- 3. 服务端唯一记账：发放/扣减只发生在云函数（打卡、注册、月赠、订单抵扣、退款返还），全部写 `points_records` 流水（02 文档 §4），余额用 `inc` 原子变更；前端任何积分参数一律忽略。
- 4. 可审计、可过期：每笔流水含 `type/refId/变更后余额`；建议积分 24 个月滚动过期（提前 30 天订阅消息提醒），过期也走流水（`type:'expire'`）。

发放规则（读 `app_config.points_rules`，运营可调）：

行为	积分	防滥用
注册	+50	一次性
每日打卡	+1/km，单日上限 50	每日仅 1 次计分（02 §5.3）
会员月赠	月100/季150/年200	定时函数发放，伴随会员有效期
订单抵扣	-100/元	抵扣上限 50%；退款时原路返还积分

6. 落地时间线（并入 04 文档 Phase 4）

步骤	责任人	耗时
备齐 § 1.2 材料 + 商户平台进件	负责人	0.5 天（审核 1-5 工作日）
打款验证 + 签约 + 绑定 AppID + 云开发绑定商户号	负责人+开发	0.5 天
wallet 云函数（unifiedOrder + payCallback + 查单/关单/退款）	开发	2 天
订单超时关单 + 每日对账定时函数	开发	1 天
会员购买接支付 + 开关切换回归（02 § 5.4 双态）	开发	1.5 天
沙箱/小额真实支付全链路验收（含重复回调、金额篡改用例）	开发+负责人	0.5 天

Sources: - [微信支付商户平台 · 小程序接入指引](#) - [微信支付商户开户意愿指引](#) - [CloudPay.unifiedOrder 官方文档](#) - [腾讯云开发：云函数接入微信支付实践](#) - [支付宝开放平台 · 支付宝小程序接入准备](#) - [微信开放社区：小程序能否使用支付宝支付](#)

06 手表/手环对接设计：蓝牙直连 + 平台授权数据采集

项目：青沐生命科技微信小程序 目标：① 小程序通过蓝牙(BLE)直连手表/心率设备，跑步时实时采集心率并随打卡入库；② 通过各厂商开放平台的 OAuth 授权（扫码/跳转授权），开通历史运动数据自动采集，替代手动打卡。参考代码仅为文档内容，实现时挂到 02 文档的 sport / user 云函数与 services 层。

0. 方案总览：两条路线，解决两类问题

	路线 A：蓝牙 BLE 直连	路线 B：平台 API 授权采集
解决什么	实时数据：跑步现场心率/实时联动打卡	历史/全量数据：跑步记录、距离、配速、睡眠、步数自动同步
原理	小程序蓝牙 API 订阅标准心率服务 (0x180D)	用户授权我方访问厂商云端数据（OAuth 2.0），服务器拉取/订阅推送
	厂商合作、服务器	现场连接设备

	路线 A：蓝牙 BLE 直连	路线 B：平台 API 授权采集
不需要		
局限	多数品牌手表默认不对第三方广播心率，需开"心率广播"模式；手环基本是私有协议连不上	各厂商要逐家申请，审核周期数天~数周；个别平台不对外开放
上线节奏	V2.0 可直接开发，无外部依赖	微信运动可立即做；华为/佳明并行申请

兼容矩阵（指导用户预期，也写进产品 **FAQ**）：

设备	路线A 实时心率	路线B 数据同步
标准蓝牙心率带/臂带（魔顿、迈金、Polar H10、佳明 HRM 等）	✓ 标准 0x180D，直连最稳	—
佳明 Garmin 手表	✓ 开启"心率广播"后可连	✓ Garmin Connect Developer Program（Health/Activity API，详见 § 2.3③）
华为手表	部分型号支持心率广播	✓ 华为 Health Kit（REST API + 订阅）
荣耀手表（2021 后新款）	部分型号支持心率广播	△ 荣耀开发者服务平台（详见 § 2.3⑤）；2020 前老款荣耀穿戴仍走华为运动健康，被 Health Kit 覆盖
高驰 COROS	✓ 开启心率广播	△ 有开放 API（对接 Strava/悦跑圈等），需商务接洽
小米手环/手表	✗ 私有协议	△ 小米开放平台可申请运动健康数据接口（详见 § 2.3④），政策以审核为准；Zepp Open Platform 需合作申请
Apple Watch	✗ 不广播	✗ HealthKit 数据不出端（无云端 API），引导用户经微信运动间接同步
微信运动（兜底，所有用户）	—	✓ wx.getWeRunData 拿 30 天步数，零申请成本

1. 路线 A：蓝牙 BLE 直连（实时心率）

1.1 前置配置

- app.json 权限声明 + 隐私接口声明（wx.openBluetoothAdapter 等已列入隐私 API，需在「小程序后台→设置→服务内容声明→用户隐私保护指引」勾选蓝牙、并在 app.json requiredPrivateInfos /隐私弹窗中覆盖）。

- Android 上搜索 BLE 需要系统定位开关打开（微信底层限制），iOS 需要系统蓝牙打开——两个失败分支都要给用户可操作的提示文案。

1.2 标准协议要点（开发必读）

- 心率服务 Service UUID：0x180D；心率测量特征值 Characteristic：0x2A37（notify）。
- 0x2A37 报文解析：第 1 字节为 flags，flags 最低位 = 0 → 心率为第 2 字节 uint8；= 1 → 心率为第 2-3 字节 uint16 小端。
- 扫描时用 services: ['180D'] 过滤，只发现广播了心率服务的设备，列表干净且省电。

1.3 连接状态机与流程

```
[未连接] → openBluetoothAdapter → startBluetoothDevicesDiscovery(180D)
  → onBluetoothDeviceFound(设备列表页) → 用户点选 → createBLEConnection
  → getBLEDeviceServices → getBLEDeviceCharacteristics(180D)
  → notifyBLECharacteristicValueChange(2A37, true)
→ [已连接·接收中] onBLECharacteristicValueChange 持续回调心率
  └─ onBLEConnectionStateChange(connected=false) → [断线] → 自动重连≤3次 → 失败提示
    └─ 页面卸载/打卡结束 → closeBLEConnection → closeBluetoothAdapter
```

1.4 参考代码（utils/ble-hrm.js 心率连接管理器）

```
// utils/ble-hrm.js — 心率设备管理器（页面只调 start/stop/onHeartRate）
const HR_SERVICE = '0000180D-0000-1000-8000-00805F9B34FB'
const HR_MEASURE = '00002A37-0000-1000-8000-00805F9B34FB'

class HrmManager {
  constructor() { this.deviceId = null; this.listeners = []; this.samples = [] }

  async start() {
    await wx.openBluetoothAdapter().catch(e => {
      throw new Error(e.errCode === 10001 ? '请先打开手机蓝牙' : '蓝牙初始化失败')
    })
    await wx.startBluetoothDevicesDiscovery({
      services: [HR_SERVICE], allowDuplicatesKey: false
    })
    wx.onBluetoothDeviceFound(res => this.emit('found', res.devices)) // 渲染设备列表
  }

  async connect(deviceId) {
    this.deviceId = deviceId
    await wx.stopBluetoothDevicesDiscovery()
    await wx.createBLEConnection({ deviceId, timeout: 10000 })
    wx.onBLEConnectionStateChange(s => { // 断线自动重连
      if (!s.connected && this.deviceId) this.reconnect()
    })
  }
}
```

```

    await wx.getBLEDeviceServices({ deviceId }) // 部分安卓需先枚举
    await wx.getBLEDeviceCharacteristics({ deviceId, serviceId: HR_SERVICE })
    await wx.notifyBLECharacteristicValueChange({
      deviceId, serviceId: HR_SERVICE, characteristicId: HR_MEASURE, state: true
    })
    wx.onBLECharacteristicValueChange(res => {
      const hr = parseHeartRate(res.value)
      this.samples.push({ hr, t: Date.now() })
      this.emit('heartRate', hr) // 页面实时显示
    })
  }

  summary() { // 打卡结束时取统计值
    const list = this.samples.map(s => s.hr)
    return list.length ? {
      avgHr: Math.round(list.reduce((a, b) => a + b) / list.length),
      maxHr: Math.max(...list), sampleCount: list.length
    } : null
  }

  async stop() {
    this.deviceId && await wx.closeBLEConnection({ deviceId: this.deviceId }).catch(() => {})
    await wx.closeBluetoothAdapter().catch(() => {})
    this.deviceId = null
  }
}

function parseHeartRate(buffer) { // 0x2A37 标准解析
  const dv = new DataView(buffer)
  const flags = dv.getUint8(0)
  return (flags & 0x01) ? dv.getUint16(1, true) : dv.getUint8(1)
}
module.exports = new HrmManager()

```

1.5 与打卡联动 (sport 页)

- 打卡表单新增「连接心率设备」入口 → 设备列表弹层 (信号强度排序) → 连接成功后表单顶部实时显示心率 (每秒刷新, >180 bpm 标红提醒)。
- 提交打卡时把 `hrm.summary()` 的 `avgHr/maxHr/sampleCount` 一并传给 `sport.checkin` ; 服务端把带设备采样的打卡标记 `source: 'ble'` , 榜单可展示"真实心率"徽标 (防作弊的正向激励)。
- 页面 `onUnload` 必须调 `hrm.stop()` 释放蓝牙。

2. 路线 B：平台 API 授权采集（自动同步运动数据）

2.1 统一授权与采集架构（各厂商共用）



为什么要"扫码/外跳"：厂商 OAuth 授权页是网页，微信小程序 web-view 只能打开业务域名内页面，无法直接内嵌厂商授权页。标准做法即上图：生成授权链接 → 二维码/复制链接到浏览器完成授权 → 回调进我方服务器。回调地址需要一个 HTTPS 域名（用云开发「云函数 HTTP 触发」或云托管 + 已备案域名）。

2.2 数据模型（新增集合）

```
// device_bindings (一人多厂商)
{ _openid, vendor: 'garmin|huawei|coros|zepp|werun',
  accessToken, refreshToken, expireAt, // 加密存储 (云函数内 AES, 密钥放环境变量)
  vendorUserId, scopes: [], status: 'active|expired|revoked',
  lastSyncAt, createdAt }

// raw_activities (厂商原始数据, 留底可追溯)
{ _openid, vendor, vendorActivityId, // vendor+vendorActivityId 唯一索引防重
  type: 'running|walking|cycling', startTime, durationSec,
  distanceMeters, avgHr, maxHr, cadence, raw: {...}, createdAt }
```

归一化与防双计：raw_activities 写入后触发归一化——type=running 且 distance≥500m 自动生成 checkin(source:'platform')；若同一用户当日已有手动打卡且时段重叠（开始时间差 < 2h），只保留设备数据并退回手动那笔的积分差额。积分上限规则与手动打卡一致（02 § 5.3）。

2.3 各平台申请方法与材料

① 微信运动 **weRun** —— 零申请，先上线

- 无需任何申请。前端 `wx.getWeRunData()` 拿加密包 → 云函数用 `cloud.getOpenData` 解密，得最近 30 天每日步数。
- 局限：只有步数（无距离/心率），适合做"步数榜"和兜底活跃数据。
- 参考代码：

```
// 前端
const { encryptedData, iv, cloudID } = await wx.getWeRunData()
await call('sport', 'syncWeRun', { weRunData: wx.cloud.CloudID(cloudID) })
// 云函数 (cloudID 自动解密)
async function syncWeRun({ OPENID, payload }) {
  const stepList = payload.weRunData.data.stepInfoList // [{timestamp, step} × 30]
  // upsert 到 raw_activities (vendor: 'werun'), 步数榜直接聚合此表
}
```

② 华为 **Health Kit**（覆盖华为/荣耀手表手环，国内占比最高，优先申请）

- 申请材料：华为开发者联盟企业实名账号（营业执照+法人信息，审核约 1-3 工作日）、应用信息、隐私政策 URL、数据使用说明（申请每一项读权限都要说明用途）。
- 步骤：开发者联盟创建应用 → 开通「Account Kit 账号服务」→ 申请「Health Kit」并勾选数据权限 scope（跑步记录、心率、步数等，权限审核按项过）→ 拿 client_id/client_secret → 实现 OAuth（授权页 → code → token）→ 调 REST API 拉数据，或注册数据订阅（数据变化时华为推送到我方回调）。
- 注意：用户须安装华为运动健康 App 且同意授权；申请到的 scope 与实际调用必须一致，多申会被驳回。

③ 佳明 **Garmin Connect Developer Program**（跑步人群浓度高，接入手册级细节）

申请：填写官方 Access Request Form，需提供法人实体信息（公司名称、官网、业务说明、预计用户量、数据用途），审核通过后签 API 协议（免费，周期约 1-2 周）。国内用户多在 Garmin Connect 中国区（佳明中国），申请时注明面向中国市场。

授权协议：OAuth 2.0 **PKCE** 模式——1. 我方服务端生成 `code_verifier`（43-128 位随机串）并算出 `code_challenge`；2. 用户访问授权 URL（带 challenge）登录 Garmin 账号、勾选权限（用户可多选权限如 Activity Export，授权后变更会经 User Permission webhook 通知我方）；3. 回调拿 code → 用 verifier 换 access_token / refresh_token。

数据获取（**Push** 模型，基本不用轮询）：在开发者门户为每类数据分别注册回调 **URL**——activities（运动记录）、dailies（每日汇总）、epochs（分钟级片段）、sleep、stress、userMetrics（VO2Max）、hrv 等；用户手表一同步，Garmin 就把数据 POST 到对应 webhook。两种通知模

式：**Push**（POST body 直接带完整数据，推荐）和 **Ping**（只给回执 URL，需再回调拉取）。历史数据用 **Backfill** 接口补拉（异步返 202，数据走 webhook 送达）。

开发要点：webhook 必须 2 秒内应答 200（先落库后异步处理）；按 vendorActivityId 去重；活动明细可取 FIT/GPX 轨迹（二期做轨迹地图再用）。

④ 小米（小米手环/手表，国内出货量最大，值得专项申请）

现状：小米健康云历史上主要面向生态链企业开放；目前可走 [小米开放平台](#) 自助申请通道，政策以实际审核为准，建议"申请 + 兜底"双轨推进。

申请路径：1. 小米开放平台注册企业开发者（营业执照等企业资料，免费，审核约 2-4 个工作日）；2. 申请「运动健康数据接口」权限：填写应用信息、数据使用说明、隐私政策 URL（审核约 3-5 个工作日），通过后获得 AppID/API 密钥；3. 实现 OAuth 授权（小米账号登录授权，纳入 § 2.1 通用框架，新增 vendors/xiaomi.js 适配器）；4. 可同步数据：步数、睡眠、心率、运动记录/轨迹（视权限审批范围）。

兜底：审核不通过或周期过长时——小米用户引导开启「小米运动健康 → 第三方数据共享 → 微信运动」，我方经 weRun 拿到步数（①已覆盖）；bind-app 页小米项显示"预约登记"收集需求量，作为与小米商务谈判的筹码。

⑤ 荣耀（与华为分家后独立生态，单独申请）

关键事实：荣耀 2020 年从华为独立后自建账号与健康生态——**2021** 年前的老款荣耀手环/手表仍绑定华为运动健康 **App**（数据走华为 Health Kit，②已覆盖，无需额外开发）；新款荣耀手表绑定「荣耀运动健康」**App**，数据在荣耀云，需单独接入。

申请路径：1. [荣耀开发者服务平台](#) 注册企业开发者（营业执照实名，流程类似华为）；2. 申请荣耀账号服务（Honor Account Kit，OAuth 用）+ 运动健康相关 Kit 权限；荣耀对穿戴生态提供 Fitness-Wear Kit 等能力，云端健康数据 **REST** 开放范围目前不如华为 **Health Kit** 完整，若所需 scope 未开放则提交商务合作申请；3. 接入纳入 § 2.1 通用框架（vendors/honor.js 适配器），授权/回调/同步逻辑与华为同构。

建议：优先级排华为/佳明之后；先上"老款走华为 Health Kit + 新款预约登记"，待荣耀云端数据开放确认后再开发适配器。

⑥ 高驰 **COROS / Zepp**（华米）—— 商务接洽通道

- COROS：有开放 API（已对接 Strava、悦跑圈等），无公开自助申请入口，走 商务邮件接洽（提供公司资质与合作方案）。
- Zepp Open Platform（dev.zepp.com）：面向合作伙伴开放数据能力，同样需合作申请。
- 产品策略：bind-app 页对未打通的厂商显示"敬请期待 + 预约登记"，统计预约量反向决定接入优先级。

2.3+ 健康数据 API 开放平台全景（备查）

回答"还有哪些健康数据开放平台"：按对本项目（微信小程序、中国市场）的可用性分层。

第一梯队（本项目已规划）：微信运动 weRun、华为 Health Kit、Garmin Connect Developer Program、小米开放平台、荣耀开发者平台。

第二梯队（国内可谈/可扩展）：

平台	数据能力	接入方式	备注
Zepp Open Platform（华米）	手环手表运动/睡眠/心率	合作申请	dev.zepp.com
高驰 COROS 开放 API	运动记录（FIT）	商务接洽	已对接 Strava/悦跑圈
OPPO 健康（OHealth）	手表运动健康数据	开发者平台/商务	开放度有限，按需接洽
vivo 健康	手表手环数据	商务接洽	无公开自助 API
Keep / 咕咚 / 悦跑圈	App 运动记录	开放平台/商务	偏 App 互联跳转，数据回流受限

第三梯队（国际平台，做出海或 App 版再考虑）：

平台	数据能力	授权	备注
Fitbit Web API（Google）	活动/睡眠/心率	OAuth2	正迁移整合至 Google 健康体系
Polar AccessLink	训练/HRV/睡眠	OAuth2	token 长期有效，接入简单
Suunto Cloud API	运动记录/步数/睡眠	OAuth2	需正式申请与 onboarding
Withings / Oura / WHOOP	体征（体重/血压/睡眠/恢复）	OAuth2	体征类数据丰富
Strava API	运动社交/活动流	OAuth2	国内跑表用户常双同步到 Strava，可作曲线数据源
Samsung Health / Health Connect	端侧聚合（Android）	端侧 SDK	需自有 App，云端 API 有限
Apple HealthKit	iPhone/Apple Watch 全量	端侧（无云 API）	数据不出端，必须有自有 iOS App
Google Fit → Health Connect	Android 聚合	端侧为主	Google Fit API 退役中，向 Health Connect 迁移

聚合服务（一接全有，按量付费）：Terra API（500+ 设备/应用源）、Thryve 等商业聚合；Open Wearables（开源自托管聚合）。优点是一次接入覆盖 Garmin/Polar/Suunto/Fitbit 等众多平台，缺点是境外服务（数据出境合规需评估）、按用户/调用收费、对国内华为/小米覆盖差。本项目结论：国内厂商直连为主，聚合服务仅在未来出海版本评估。

2.4 服务端参考代码（OAuth 回调 + 定时同步）

```
// 云函数 device-oauth (HTTP 触发) /oauth/callback
async function callback(query) {
  const { code, state } = query
  const { openid, vendor } = await verifyState(state) // state 一次性、5分钟有效
  const token = await exchangeToken(vendor, code) // code→access_token
  await upsert('device_bindings', { _openid: openid, vendor }, {
    accessToken: encrypt(token.access_token),
    refreshToken: encrypt(token.refresh_token),
    expireAt: Date.now() + token.expires_in * 1000, status: 'active'
  })
  return htmlPage('绑定成功，请返回小程序') // 回调落地页
}

// 云函数 device-sync (定时触发器：每小时；佳明走 push 则此函数只兜底)
async function syncAll() {
  const bindings = await db.collection('device_bindings')
    .where({ status: 'active' }).get()
  for (const b of bindings.data) {
    if (b.expireAt < Date.now()) await refreshToken(b) // 过期先刷新
    const acts = await fetchActivities(b, b.lastSyncAt) // 各厂商适配器
    for (const a of acts) await saveRawActivity(b._openid, b.vendor, normalize(a))
    await markSynced(b)
  }
}

// vendors/garmin.js、vendors/huawei.js 实现 fetchActivities/exchangeToken 适配器，
// 新增厂商只加一个适配器文件，主流程不动。
```

2.5 bind-app 页改造（替换现在的假绑定）

- 列表项三态：未绑定（按钮“去授权”→弹二维码/复制链接）| 已绑定（显示最近同步时间 + “立即同步” + “解绑”）| 敬请期待（预约登记）。
- 解绑：删 token、status→revoked，并调厂商 revoke 接口（有则调）。
- 全模块受 `feature_flags.bindApp` 开关控制，厂商逐个开（flags 细化为 `bindApp: {werun:true, huawei:false, garmin:false}`）。

3. 任务拆解（并入 04 文档，作为 Phase 6）

ID	任务	前置	工作量	验收标准
T6-1	微信运动接入（getWeRunData + 解密入库 + 步数榜）	Phase 2	1.5 天	30 天步数入库；同账号重复同步不重复计
T6-2	BLE 心率管理器（utils/ble-hrm.js）+ 设备列表弹层	—	2 天	心率带真机连接、断线重连、页面退出释放
T6-3	打卡联动心率（source:'ble'，avg/max 入库与展示）	T6-2	1 天	带心率打卡在榜单显示徽标
T6-4	OAuth 通用框架（HTTP 触发 + state 防伪 + token 加密存储 + 适配器模式）	备案域名	2 天	模拟厂商完成全流程
T6-5	华为 Health Kit 申请（负责人）+ 适配器 + 数据订阅	T6-4、华为审核	2.5 天	华为手表跑步记录自动生成打卡
T6-6	佳明申请（负责人）+ 适配器 + 分类型 webhook + Backfill 补拉	T6-4、佳明审核	2.5 天	佳明新活动 10 分钟内入库；历史 30 天可补拉
T6-7	归一化与防双计（手动打卡 vs 设备数据去重退分）	T6-5/6	1 天	重叠时段不重复计分
T6-8	bind-app 页重做（三态 + 预约登记 + 分厂商开关）	T6-4	1 天	假绑定代码全部删除
T6-9	小米开放平台申请（负责人，企业入驻+数据权限）+ vendors/xiaomi.js 适配器	T6-4、小米审核	2 天	小米手环运动记录自动生成打卡；审核被拒则走微信运动兜底
T6-10	荣耀接入：老款确认走华为 Health Kit；荣耀开发者平台入驻 + 新款数据开放评估（视开放范围决定是否开发 vendors/honor.js）	T6-5	1 天（评估）+ 2 天（若开发）	老款荣耀手表数据经华为链路入库；新款给出可/不可接结论

风险：① 厂商审核周期不可控（华为权限逐项审、佳明 1-2 周）→ 提前由负责人发起申请，与开发并行；② 蓝牙在部分安卓机型兼容性差 → 准备 3+ 真机回归清单；③ token 泄漏风险 → 加密存储 + 集合权限"仅云函数可读写" + 定期轮换密钥。

Sources: - [微信开放文档](#) · [蓝牙 \(Bluetooth\)](#) - [华为 Health Kit 服务介绍](#) · [接入流程](#) - [Garmin Connect Developer Program](#) · [Health API](#) · [Activity API](#) · [OAuth2 PKCE 规范](#) · [Access](#)

07 国内菜谱与饮食营养 API 选型

项目：青沐生命科技微信小程序 用途：为「餐饮/健康饮食」模块（03 文档 content type=food）和未来"运动 + 饮食管理"闭环提供数据源。本文盘点国内可用的菜谱/营养/识别类 API，给出选型建议、缓存落地设计与参考代码。

1. 业务场景 → 需要什么 API

产品场景	需要的能力	对应 §2 分类
食谱内容（跑前/跑后怎么吃、健康食谱推荐）	菜谱大全（菜名、食材、步骤、图片）	A 菜谱内容
食物热量/营养查询（"一碗米饭多少大卡"）	营养成分数据库	B 营养成分
拍照识别菜品记饮食（进阶玩法）	AI 菜品图像识别	C AI 识别
扫包装条码看配料/热量	条码→商品/配料	D 条码查询
专业营养内容/品牌联名	内容商务合作	E 合作通道

2. 国内可用 API 盘点

A. 菜谱内容 API（按成熟度排序）

平台	数据规模	能力	计费
聚合数据·菜谱大全	10 万+ 道，按蛋/奶/面/蔬果/肉/水产等分类	关键词/分类查询，含图文步骤	免费额度 + 按次付费
极速数据·菜谱	1 万+ 道	分类树、关键词搜索、主料辅料、做法步骤	免费额度 + 套餐
天行数据 TianAPI·菜谱查询	数万道	随机菜谱、按分类检索	会员套餐制，单价低
免费 API 聚合站（free-api 等收录的菜谱接口）	不稳定	仅原型期临时用	免费

三家正规商用平台（聚合/极速/天行）均为标准 HTTPS+JSON、需注册企业账号申请 AppKey。菜谱属于内容数据，注意版权条款——选购时确认"允许商用展示"，页面保留来源标注。

B. 食物营养成分 / 热量 API

平台	数据规模	能力
天行·营养成分表	约 2000 种常见食物	每 100g 营养成分与微量元素、按成分排序检索
RollTools 等食物热量 API	数千种	食物分类/列表/搜索/详情四接口
《中国食物成分表》（标准版/全国代表值）	权威纸质/数据授权	无公开 API；可购买数据授权后自建库（量大且需权威性时的正解）
薄荷健康食物库	50 万+ 种（含包装食品）	国内最全，但无公开 API ，走 E 类商务合作
FatSecret API（国际）	全球库，有中文数据	OAuth 接入，免费层可用；国内访问稳定性与合规需评估

C. AI 菜品/食材识别

平台	能力	说明
百度 AI·菜品识别	识别 9000+ 菜品，返回菜名、置信度、卡路里、百科信息；支持自建菜品图库	国内最成熟，有免费测试额度，之后次数包/按量付费；接口 <code>image-classify/v2/dish</code>
天行·食物营养识别	传图识别近 2000 种食物并返回营养成分	识别+营养一步到位，规模小于百度
腾讯云/阿里云通用图像识别	通用标签含菜品类目	无专门菜品接口，准确率不如百度专项

D. 食品条码查询

- 聚合数据、阿里云云市场等有「商品条码查询」API（条码→商品名/厂商/规格，部分含配料表）；数据完整度参差，包装食品营养以实拍营养成分表为准，条码 API 只做辅助填充。

E. 内容/数据商务合作（API 之外的正路）

- 薄荷健康：50 万食物库 + 营养师内容，国内健康饮食数据第一梯队，走商务授权（青沐"生命科技"定位与其调性契合，建议接洽）。
- 下厨房 / 豆果美食：菜谱内容头部，无公开自助 API，内容授权/联合运营需商务谈。

- 中国营养学会：权威膳食指南内容授权，适合做"科学背书"型内容。

3. 选型建议（结合本项目）

1. **V1**（**content food** 模块上线即用）：不接 API。食谱/饮食内容由运营在 `contents (type=food)` 录入 10-30 篇精选图文（跑者餐单、赛前碳水攻略），成本为零、可控、无版权风险。
2. **V2**（饮食查询功能）：接一家菜谱 API（推荐聚合数据，规模最大）+ 天行营养成分表（热量查询），均经云函数代理并缓存（§4），月成本预计百元级。
3. **V2.5**（拍照记饮食，差异化亮点）：百度菜品识别 + 营养库联动——拍照→菜名+卡路里→写入用户饮食日记，与运动消耗对比形成"摄入 vs 消耗"闭环（大健康产品的核心粘性功能）。
4. **V3**（数据自主化）：若饮食功能被验证为核心，购买《中国食物成分表》数据授权自建营养库 + 接洽薄荷健康，摆脱按次计费与第三方依赖。

4. 落地设计：云函数代理 + 缓存（控制成本的关键）

小程序 → `content` 云函数(`action=foodSearch`) → ① 查 `food_cache` 集合命中即返回
└─ ② 未命中→调第三方API→写缓存→返回

- 第三方 API 按次计费，绝不允许小程序直连（AppKey 会泄漏，也无法缓存）。AppKey 存云函数环境变量。
- 缓存集合 `food_cache`：{ `keyword`, `source: 'juhe|tianapi|baidu'`, `payload`, `hitCount`, `expiredAt` }，菜谱类缓存 30 天、营养成分缓存 180 天（基本不变）、识别结果按图片 hash 缓存。
- 限流：每用户每日查询/识别次数上限（如 20 次）写在 `app_config`，防止恶意刷量刷爆 API 账单。

参考代码（`content` 云函数新增两个 action）：

```
// foodNutrition：营养成分查询（带缓存）
async function foodNutrition({ payload }) {
  const { keyword } = payload
  const hit = await db.collection('food_cache')
    .where({ keyword, source: 'tianapi', expiredAt: _.gt(Date.now()) }).get()
  if (hit.data.length) return hit.data[0].payload

  const res = await axios.get('https://apis.tianapi.com/nutrient/index', {
    params: { key: process.env.TIANAPI_KEY, word: keyword }
  })
  if (res.data.code !== 200) throw { code: 502, message: '营养库查询失败' }
  await db.collection('food_cache').add({ data: {
    keyword, source: 'tianapi', payload: res.data.result,
```

```

    expiredAt: Date.now() + 180 * 864e5, createdAt: new Date()
  })
  return res.data.result
}

// dishRecognize：百度菜品识别（图片先传云存储，云函数取临时链接转 base64）
async function dishRecognize({ OPENID, payload }) {
  await checkDailyQuota(OPENID, 'dishRecognize', 20) // 限流
  const token = await getBaiduToken() // access_token 缓存30天
  const res = await axios.post(
    'https://aip.baidubce.com/rest/2.0/image-classify/v2/dish',
    qs.stringify({ image: payload.imageBase64, top_num: 3 }),
    { params: { access_token: token } })
  const best = res.data.result?.[0]
  if (!best) throw { code: 404, message: '未识别出菜品' }
  return { name: best.name, calorie: best.calorie, probability: best.probability }
}

```

5. 任务与风险

任务	工作量	验收
T7-1 V2：聚合菜谱 + 天行营养接入（云函数代理+缓存+限流）	2 天	同一关键词第二次查询不产生外部调用
T7-2 V2：food 页加"热量查询"入口	1 天	查询"米饭"返回每100g营养成分
T7-3 V2.5：百度菜品识别 + 饮食日记（meals 集合）+ 摄入/消耗对比卡片	3 天	拍照→菜名卡路里入日记；首页显示今日摄入vs运动消耗

风险：① 第三方 API 停服/涨价 → 缓存层让切换供应商只改适配器；② 内容版权 → 商用条款留档，页面标注来源；③ 健康声明合规 → 营养建议类文案加"仅供参考，不构成医疗建议"，避免医疗化表述（小程序审核红线）。

Sources: - [聚合数据·菜谱大全 API](#) · [极速数据·菜谱 API](#) - [天行数据·菜谱查询](#) · [天行·营养成分表](#) · [天行·食物营养识别](#) - [百度 AI 菜品识别](#) · [接口文档](#) - [FatSecret API 实践参考](#)

08 菜谱数据接入采集方案 + 律动平台对接设计

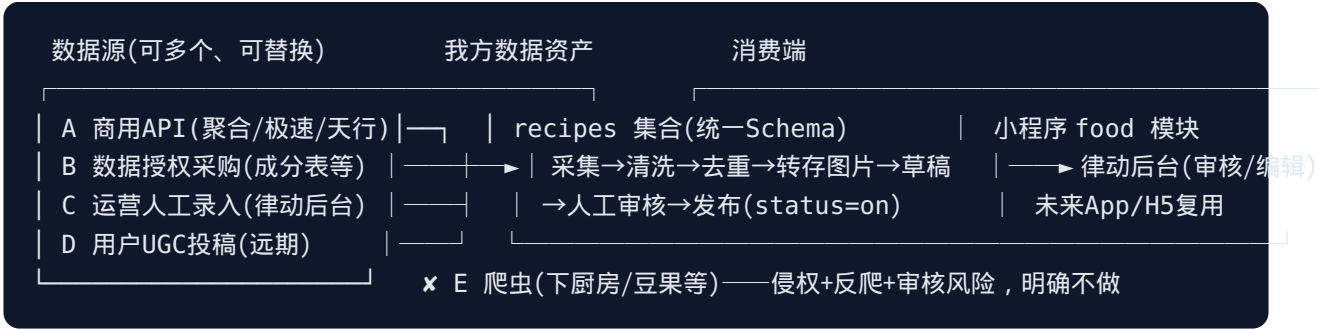
项目：青沐生命科技微信小程序 前提（已确认）：律动平台 = 公司内部运营/管理后台，已有 HTTP API；对接为双向——小程序→律动（用户与运动数据、订单/积分），律动→小程序（内容

下发），并做用户账号打通。本文承接 07 文档（API 盘点），回答"数据怎么进来、怎么管、怎么和律动互通"。

Part 1 菜谱数据接入采集综合分析

1.1 核心结论：不做"实时透传"，做"采集入库制"

第三方菜谱 API 按次计费、格式互不相同、随时可能停服。正确姿势是把它们当作一次性/周期性的数据源，采集进自己的库，小程序只读自己的库：



四种来源对比与采用策略：

来源	成本	质量	版权	策略
A 商用 API 批量采集	按次付费，万级数据约数百元	中，需清洗	按购买条款商用，留源标注	主力：V2 一次性采集建库 + 每月增量
B 数据授权采购	一次性数千~数万元	高（权威）	清晰	营养成分表走这条（07 § 3.4）
C 运营录入（律动后台）	人力	最高（品牌调性）	自有	精选内容：跑者餐单等 30-50 篇自产
D 用户投稿	低	不稳，需审核	用户授权协议	远期社区化再开

1.2 统一数据模型（recipes 集合）

所有来源归一到一个 Schema，消费端与来源解耦：

```
{
  _id, title, coverFileID, // 图片一律转存云存储（防外链失效/盗链限制）
  category: 'breakfast|prerun|postrun|lowcal|homedish|...', // 我方业务分类（非源分类）
  tags: ['高蛋白','补碳','减脂'],
  ingredients: [{ name:'鸡胸肉', amount:'200g', isMain:true }],
```



```

steps: [{ order:1, text:'...', imageFileID }],
nutrition: { calorie, protein, fat, carb, per:'100g'|'serving' } | null, // 关联营养库
durationMin, difficulty: 1-5, servings,
source: { type:'api|licensed|editorial|ugc', vendor:'juhe', vendorId:'8512', license:'商用授
audit: { status:'draft|reviewing|on|off', reviewer, reviewedAt },
stats: { views, favorites },
fingerprint, // 去重指纹（见1.3）
createdAt, updatedAt
}

```

1.3 采集管道（ETL，云函数实现）

recipe-ingest 云函数（手动触发批量 / 定时增量）

- ① 拉取：按分类遍历源 API（限速 ≤2 req/s，断点续采：记录游标到 ingest_jobs）
- ② 映射：vendors/juhe-recipe.js 等适配器 → 统一 Schema（字段映射表见下）
- ③ 清洗：
 - 单位标准化（"适量/少许"保留原文；克/毫升统一）
 - 文本过滤：敏感词、医疗化表述（"治疗""降血压"→拦截人工处理）
 - 步骤完整性校验（无步骤/无主料的丢弃，记入 ingest_rejects）
- ④ 去重：fingerprint = sha1(normalize(title) + '|' + 主料排序拼接)
命中已有 → 跳过或按 updatedAt 更新；跨源重复以先入库者为准
- ⑤ 图片转存：源图 URL 下载 → 压缩(≤300KB) → 云存储 → 替换为 fileID
- ⑥ 落库：audit.status = 'draft'
- ⑦ 审核发布：运营在律动后台改分类/标签/排版 → status='on' 后小程序可见

字段映射表（开发对照用，节选）：

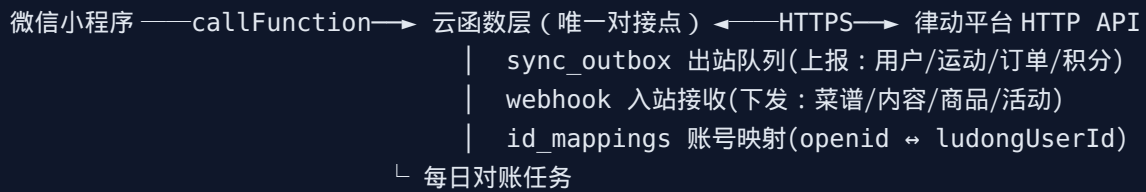
统一字段	聚合数据	极速数据	天行
title	title	name	cp_name
ingredients	yuanliao/tiaoliao	material[]	yuanliao
steps	zuofa(分步)	process[].pcontent	zuofa
coverFileID	albums[0]	pic	—

1.4 运营与质量闭环

- 量级建议：V2 首批采集 3000-5000 道（覆盖常见家常+运动餐分类即可，不求十万级——没人翻得完，审核也审不完）；精选 50 篇编辑内容置顶。
- 增量：每月增量采集一次新分类；用户搜索无结果的关键词记录到 search_misses，作为下批采集清单（按需采集，最省钱）。
- 下线机制：用户举报/审核抽查 → status='off'，保留数据可追溯。
- 合规：详情页尾部固定"内容来源：XX数据，仅供参考，不构成医疗建议"。

Part 2 律动平台对接设计

2.1 总体拓扑与原则



原则：1. 小程序永不直连律动——所有交互经云函数，律动地址/密钥放云函数环境变量。2. 双向都走事件+幂等：每条数据带全局唯一 `eventId`，重发不重复入账（at-least-once 投递）。3. 各自数据库为各自主权：运动明细以小程序侧为准（source of truth），内容以律动侧为准，订单以小程序侧为准、律动只读镜像；不做双写同一业务，避免脑裂。

2.2 服务间认证（与律动后端约定）

- 传输：HTTPS；如律动在内网，经公司网关暴露专用域名+IP 白名单（云函数出口 IP 段在云开发控制台可查）。
- 签名：HMAC-SHA256。请求头 `X-App-Id`、`X-Timestamp`、`X-Nonce`、`X-Signature` = `HMAC(secret, method+path+timestamp+nonce+sha256(body))`；时间窗 ±5 分钟防重放。双向调用各发一对 `AppId/Secret`。
- 敏感字段（手机号）传输前脱敏或字段级加密，按最小必要原则给。

2.3 账号打通（id_mappings）

绑定流程（一次性）：
小程序「我的→绑定律动账号」→ 输入手机号 → 律动 API 发短信验证码 → 校验通过
→ 律动返回 `ludongUserId` → 写 `id_mappings { _openid, ludongUserId, boundAt }`
→ 此后所有上报/下发都带双方 ID
未绑定用户：运动数据照常上报（只带 `openid`），律动侧建影子账号，待绑定后合并

2.4 接口清单（与律动后端对齐的契约草案）

方向 **A**：小程序 → 律动（云函数出站，批量+实时结合）

接口	触发	载荷要点
POST /open/v1/users/upsert	注册/资料变更（实时）	eventId, openid, ludongUserId?, profile 脱敏字段
	定时 5 分钟批量	

接口	触发	载荷要点
POST /open/v1/checkins/ batch		events[] : 打卡/设备同步记录 (distance、avgHr、source)
POST /open/v1/orders/ sync	订单状态变更 (实时)	eventId, orderId, status, items, payAmount
POST /open/v1/points/ sync	积分流水 (定时批量)	points_records 增量 (游标=createdAt)

方向 **B** : 律动 → 小程序 (内容下发)

接口	模式	说明
POST <云函数HTTP触发>/webhook/ludong	律动有变更即推	type: recipe/content/product/banner + 数据体 ; 验签后 upsert 到 recipes/contents/products , status 直接进 'on' (律动后台已审核)
GET /open/v1/contents/changes?cursor=	每小时定时拉 (兜底)	webhook 丢失时按 updatedAt 游标补拉 , 保证最终一致

菜谱采集管道 (Part 1) 与律动的关系 : 采集入库后 **audit** 流转放在律动后台做——律动加"菜谱审核"模块读写 recipes 集合 (经方向 B 同样的 API 通道) , 运营在熟悉的后台完成审核/编辑/上下架 , 小程序侧零运营界面。

2.5 可靠性设计

- 出站队列 **sync_outbox** : 业务写库成功时同事务写一条 outbox 记录 (pending) → 定时函数批量投递 → 律动应答 200 + ackEventId 置 done ; 失败指数退避重试 (1/5/30 分钟) , 超 24h 转 dead 并告警。业务永不因律动宕机而失败。
- 入站幂等 : webhook 按 eventId 查重表 inbound_events , 处理过直接 200。
- 对账 : 每日 03:00 双方交换前一日计数摘要 (用户数/打卡数/订单数/积分净额) , 不一致触发明细比对任务并告警到企业微信。
- 监控 : outbox 积压量、webhook 失败率写入 app_config 可视化 (律动后台展示)。

2.6 参考代码 (云函数侧)

```
// utils/ludong-client.js — 出站签名客户端
const crypto = require('crypto')
async function callLudong(method, path, body) {
  const ts = Date.now().toString(), nonce = crypto.randomUUID()
  const bodyHash = crypto.createHash('sha256').update(JSON.stringify(body || {})).digest('hex')
```

```

const sign = crypto.createHmac('sha256', process.env.LUDONG_SECRET)
  .update([method, path, ts, nonce, bodyHash].join('\n')).digest('hex')
return axios({ method, url: process.env.LUDONG_BASE + path, data: body,
  headers: { 'X-App-Id': process.env.LUDONG_APPID, 'X-Timestamp': ts,
    'X-Nonce': nonce, 'X-Signature': sign }, timeout: 8000 })
}

// 云函数 sync-outbox (定时触发：每5分钟)
async function flushOutbox() {
  const batch = await db.collection('sync_outbox')
    .where({ status: 'pending', nextRetryAt: _.lte(Date.now()) })
    .orderBy('createdAt', 'asc').limit(50).get()
  for (const e of batch.data) {
    try {
      await callLudong('POST', e.path, { eventId: e._id, ...e.payload })
      await db.collection('sync_outbox').doc(e._id).update({ data: { status: 'done' } })
    } catch (err) {
      const retry = (e.retryCount || 0) + 1
      await db.collection('sync_outbox').doc(e._id).update({ data: {
        retryCount: retry, status: retry >= 6 ? 'dead' : 'pending',
        nextRetryAt: Date.now() + Math.min(retry * retry, 36) * 5 * 60e3, lastError: err.message
      }})
    }
  }
}

// 云函数 webhook-ludong (HTTP 触发) — 内容下发接收
exports.main = async (event) => {
  const req = parseHttp(event)
  if (!verifyHmac(req)) return { statusCode: 401 }
  const { eventId, type, data } = JSON.parse(req.body)
  if (await seen(eventId)) return { statusCode: 200 } // 幂等
  const handler = { recipe: upsertRecipe, content: upsertContent,
    product: upsertProduct }[type]
  if (!handler) return { statusCode: 400 }
  await handler(data); await markSeen(eventId)
  return { statusCode: 200, body: JSON.stringify({ ack: eventId }) }
}

```

2.7 任务拆解 (Phase 7)

ID	任务	前置	工作量	验收
T8-1	与律动后端联合定稿 § 2.4 接口契约 + 密钥交换	律动团队	1 天 (会议 + 文档)	双方签认契约文档
T8-2	ludong-client 签名客户端 + sync_outbox 队列 + 定时投递	T8-1	2 天	律动停机 1 小时后恢复, 数据零丢失自动补投

ID	任务	前置	工作量	验收
T8-3	webhook 接收 + 内容/菜谱/商品 upsert + 兜底拉取	T8-1	1.5 天	律动改一篇菜谱，小程序 1 分钟内可见
T8-4	账号绑定流程（手机号验证 + id_mappings + 影子账号合并）	T8-1	1.5 天	绑定后历史打卡在律动侧归到正确用户
T8-5	菜谱采集管道（recipe-ingest + 三源适配器 + 去重清洗 + 图片转存）	07 文档选型	3 天	首批 3000 道入库为 draft，重复率 <1%
T8-6	律动后台菜谱审核模块（律动团队开发，我方提供 API）	T8-3/5	律动侧	审核→发布→小程序可见全链路通
T8-7	每日对账 + 积压告警	T8-2	1 天	人为制造差异能在次日告警

风险：① 律动 API 变更无版本管理 → 契约文档 + /v1 路径版本化；② 手机号绑定涉及个人信息共享 → 隐私政策列明"与公司内部系统共享"，绑定页单独勾选同意；③ 采集图片转存量较大 → 云存储费用预估（5000 道 × 2 图 × 300KB ≈ 3GB，费用可忽略）。

09 代码优化清单（重扫）

项目：青沐生命科技微信小程序 重扫日期：2026-06-11 | 视角：优化（性能 / 包体 / 可维护性 / 代码去重），区别于 01 号文档的「缺陷审查」。重扫结论：自上次审查以来代码无实质变化（仍 15 页面 / 13 云函数；sitemap.json、云函数 package.json 仍缺失；新增一个空 images/ 目录与 smart-agent.wxss）。故 01/04 号文档结论依旧有效。本文在其基础上，给出纯优化向的可直接落地改法（含 before/after），不重复罗列已知缺陷。

△ 优先级提示：本项目正按 02 号文档做架构重构。重构会重写大部分页面，因此本文的优化项分两类标注：◆ [重构顺带做]——在重写该模块时一并采用，无需单独排期；○ [独立可做]——即使不重构也值得现在改，投入小见效快。

0. 优化机会总览（按收益/成本排序）

#	优化项	类型	收益	成本	标注
O-1	6 个雷同"列表+预订"页合并为模板页	代码去重	删约 600 行重复代码	中	◆
O-2		可维护性	改一处生效全局	中	◆

#	优化项	类型	收益	成本	标注
	callFunction 统一封装（现散落 11 文件 17 处）				
O-3	group.js 拆分（657 行 / 17 次 setData / 3 职责）	性能+可维护	首屏快、易维护	中	◆
O-4	setData 瘦身（只传变化字段、大列表分页）	运行性能	渲染卡顿↓	小	○
O-5	分包加载（subPackages）	启动性能	主包体积↓、首开快	小	○
O-6	app.wxss 3565 行拆分 + 设计变量	可维护性	样式可控、改色一处	中	◆
O-7	mock 数据移出 data（31 处硬编码）	可维护性	data 轻、逻辑清晰	小	◆
O-8	公共逻辑抽 utils（会员等级/格式化/openid）	代码去重	消除多处重复	小	◆
O-9	长列表 wx:key 与渲染优化	渲染性能	列表滚动流畅	小	○
O-10	资源与构建（图片、debug、console.log）	包体+规范	体积↓、上线就绪	小	○

1. O-1 雷同页面合并（最大收益）◆

现状：food / scenic / hotel / marathon / marathon-expo / rural-support 六个页面结构 95% 相同——都是「data 里塞一个硬编码数组 + loadX() 空壳 + registerX/bookX/orderX/buyX() 弹确认框 + saveX() 调用一个不存在的云函数」。仅 food.js 就调 save-food-order（又一个不存在的云函数，与 01 文档 P0-5 同类）。

六页 .js 合计约 700 行，去重后一个模板页约 120 行，净删约 600 行。

before（每页一份，six times）：

```
// pages/food/food.js — 和 hotel/scenic/marathon... 仅字段名不同
Page({
  data: { foods: [ {id:1,name:'全聚德烤鸭',price:198,...}, /* 5条硬编码 */ ] },
  onLoad(){ this.loadFoods() },
  loadFoods(){ console.log('加载餐饮列表') }, // 空壳
  orderFood(e){ /* showModal → setTimeout → saveOrder → Toast */ },
  saveOrder(food){ wx.cloud.callFunction({ name:'save-food-order', /* 不存在 */ }) }
})
```

after (一个模板页, 按 **type** 驱动; 对接 02 文档 **content** 云函数) :

```
// pages/content-list/content-list.js
import { content } from '../../services/index'
Page({
  data: { type: '', list: [], loading: true },
  async onLoad({ type }) { // type=food|scenic|hotel|marathon|...
    wx.setNavigationBarTitle({ title: TYPE_TITLE[type] })
    this.setData({ type, list: await content.list({ type }), loading: false })
  },
  async onEnroll(e) { // 统一"报名/预订意向"
    const id = e.currentTarget.dataset.id
    await content.enroll({ id }) // 走真实云函数, 登记意向
    wx.showToast({ title: '已提交, 工作人员将联系您' })
  }
})
```

配 `content-list.wxml` 一套模板 + `app.json` 用 `?type=` 传参 (首页宫格已是 `goToMarathon/goToHotel...`, 改成统一跳转即可)。6 个 `.js` + 6 个 `.wxml` → 2 个文件。

2. O-2 callFunction 统一封装 ◆

现状: `wx.cloud.callFunction` 散落 11 个文件、17 处, 每处各写一套 `success/fail` + `console.log` + `userInfo.openId || 'test_openid'`, 风格还混用回调与 `async/await`。改个超时、加个全局 `loading`、统一错误提示都得改 17 处。

after (**services** 层, 全项目唯一出口) :

```
// services/api.js
const app = getApp()
export function call(name, action, payload = {}, { loading = true } = {}) {
  loading && wx.showLoading({ title: '加载中', mask: true })
  return wx.cloud.callFunction({ name, data: { action, payload } })
    .then(res => {
      const r = res.result
      if (r.code !== 0) throw r // 统一错误约定
      return r.data
    })
    .catch(err => { wx.showToast({ title: err.msg || '网络异常', icon: 'none' }); throw err })
    .finally(() => loading && wx.hideLoading())
}
// services/index.js — 领域封装
export const content = {
  list: p => call('content', 'list', p),
  enroll: p => call('content', 'enroll', p),
}
```

```
}  
export const sport = { checkin: p => call('sport', 'checkin', p), /* ... */ }
```

页面侧从十几行回调塌缩成一行 `await sport.checkin({...})`。openid 永不在前端出现（02 文档 P0-2）。

3. O-3 group.js 拆分 ◆

现状：`group.js` 657 行、17 次 **setData**，一页混了三件事：跑群绑定、打卡、周/月/年汇总展示，且含 4 大段硬编码 mock（成员、周/月/年 summary）。是全项目最重的页面。

改法（与 02 文档目录一致）：- 拆成 `sport`（打卡 + 我的群列表）+ `group-detail`（榜单/汇总），各自约 150 行。- 三段 summary mock 删除，改 `sport.groupRanking({period})` 服务端聚合。- `showWeekSummary/showMonthSummary/showYearSummary` 三个几乎一样的函数 → 一个 `loadRanking(period)`。

before :

```
showWeekSummary(){ this.setData({ currentSummary: { /*硬编码周*/ }, summaryPeriod: '...' }) }  
showMonthSummary(){ this.setData({ currentSummary: { /*硬编码月*/ }, summaryPeriod: '...' }) }  
showYearSummary(){ this.setData({ currentSummary: { /*硬编码年*/ }, summaryPeriod: '...' }) }
```

after :

```
async loadRanking(e){  
  const period = e.currentTarget.dataset.period // week|month|year  
  this.setData({ currentSummary: await sport.groupRanking({ groupId: this.data.groupId, period  
  })  
}
```

4. O-4 setData 瘦身 ○（不重构也该改）

微信 setData 是逻辑层→渲染层跨线程通信，传得越大越频繁越卡。两条通用规则：

1. 只传变化的字段，别整对象覆盖。- before：`this.setData({ paymentMethods })`（钱包页整个对象回写）- after：`this.setData({ ['paymentMethods.wechat']: true })`（路径更新，只传变化项）
2. 大列表不要一次性灌进 **data**。商城 `statistics.js` 一次塞 8 条带长 `description` 的商品；真实数据应分页 `concat` 追加，配合 `onReachBottom`。
3. 避免在循环里 **setData**；先聚合成一个对象再一次性 set。

5. O-5 分包加载 ○

现状：app.json 15 个页面全在主包。商城详情、内容五页、智能体等首开用不到，却都算进主包体积，拖慢冷启动。

after：

```
// app.json
{
  "pages": ["pages/index/index", "pages/sport/sport", "pages/mall/mall", "pages/mine/mine"],
  "subpackages": [
    { "root": "pkg-content", "pages": ["content-list/content-list", "content-detail/content-detail"],
      { "root": "pkg-mall", "pages": ["product-detail/product-detail", "order-confirm/order-confirm"] },
  ],
  "preloadRule": { "pages/index/index": { "packages": ["pkg-content"] } } // 首页空闲时预载
}
```

主包只留 4 个 tab 页，其余按需加载 + 预载，冷启动更快。

6. O-6 app.wxss 拆分 + 设计变量 ◆

现状：app.wxss 3565 行、545 条 class 全在全局，主色硬编码微信绿 #1aad19（非青沐品牌色），改色要全局搜替换。

after：

```
/* app.wxss 顶部：设计令牌，约束在 300 行内只放通用类 */
page {
  --brand: #0FAF8E; --brand-dark: #0B8C72; --brand-bg: #E6F7F3;
  --text-1: #1f2937; --text-2: #6b7280; --radius: 12rpx;
}
.btn-primary { background: var(--brand); color: #fff; border-radius: var(--radius); }
```

- 页面专属样式下沉到各自 pages/x/x.wxss；全局只留 token + 通用类。
- 改品牌色从"全局替换 #1aad19"变成"改一个变量"。

7. O-7 ~ O-10 其余优化（小成本）

O-7 mock 移出 data ◆：group/statistics/food/hotel/marathon/scenic/... 共 31 处硬编码业务字段堆在 data，使初始 data 臃肿且与真实数据混淆。重构期一律改为接口拉取；过渡期可暂存 constants.js。

O-8 公共逻辑抽 utils ◆：- 会员等级→权益映射在 group.js 和 settings.js 各写一份（01 文档 P1-9），抽 utils/member.js 单一来源。- 配速/距离/日期格式化散落多页 → utils/format.js。- userInfo.openId || 'test_openid' 14 处 → 随 O-2 一并消除。

O-9 列表渲染 ○：wx:for 用稳定业务 id 作 wx:key（现多处用 index，数据变动会整列重渲）；长内容列表配合 O-4 分页。

O-10 资源与构建 ○：- app.json 删 "debug": true；补 sitemap.json（否则构建告警）。- 全量删除 console.log（40+ 处）；云函数保留 console.error。- 空 images/ 目录补 tabBar 图标或删除；emoji 图标换切图（跨机型渲染不一致）。- 补 .gitignore（project.private.config.json、.DS_Store、node_modules/）。

8. 落地建议

1. 不要为优化单独重写：◆ 项随 02/04 文档的模块重构顺带完成，不额外排期。
2. 现在就能做的 ○ 项（O-4/O-5/O-9/O-10）投入小、与重构不冲突，可作为重构前的"热身 PR"，约 1-1.5 人天。
3. 验收：主包体积下降（可在开发者工具"代码依赖分析"看）、首页冷启动时间下降、grep -r "callFunction" pages 仅命中 services 层、grep -rn "console.log\|test_openid" 归零。

配套：本目录 01-code-review.md（缺陷）、02-architecture.md（目标结构）、04-task-breakdown.md（任务表）。本文 ◆ 项已隐含在 04 的 Phase 1-3，○ 项可并入 Phase 0。